JET BRAINS Kotlin

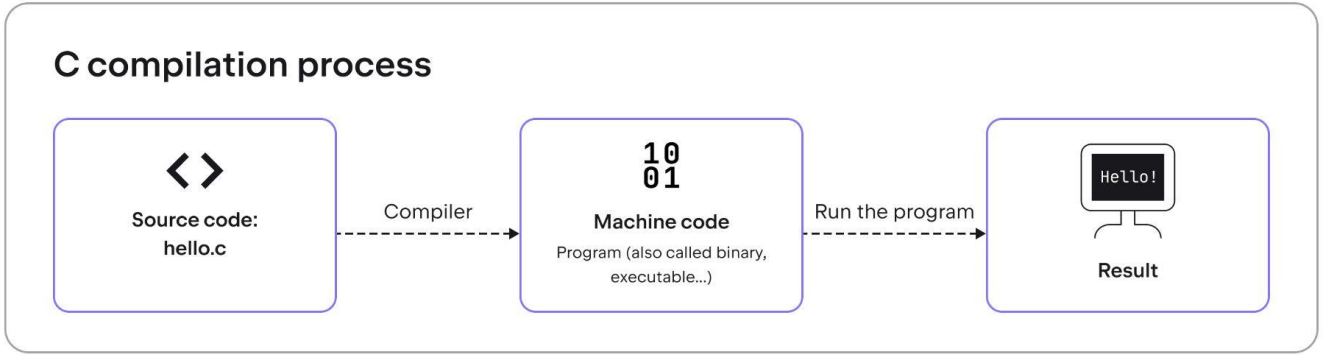
Виртуальная машина Java и Компилятор Kotlin

@kotlin | Developed by JetBrains

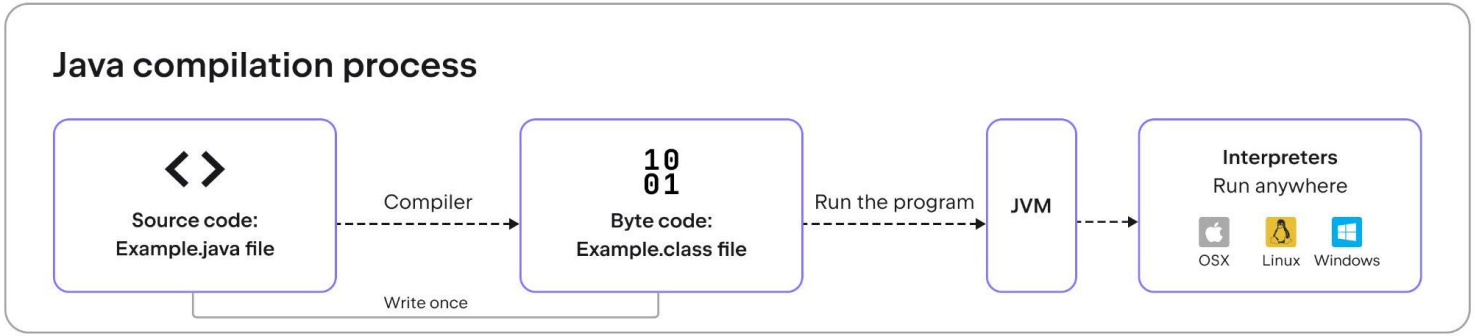
Язык Java

- Создан в 1995.
- ООП ориентированный язык со строгой типизацией.
- Есть Just-in-time (JIT) компиляция.
- Использует Java Virtual Machine (JVM).
- Есть сборщик мусора (GC), который очищает выделенную (аллоцированную) память автоматически.

Процесс компиляции – Java vs C



VS

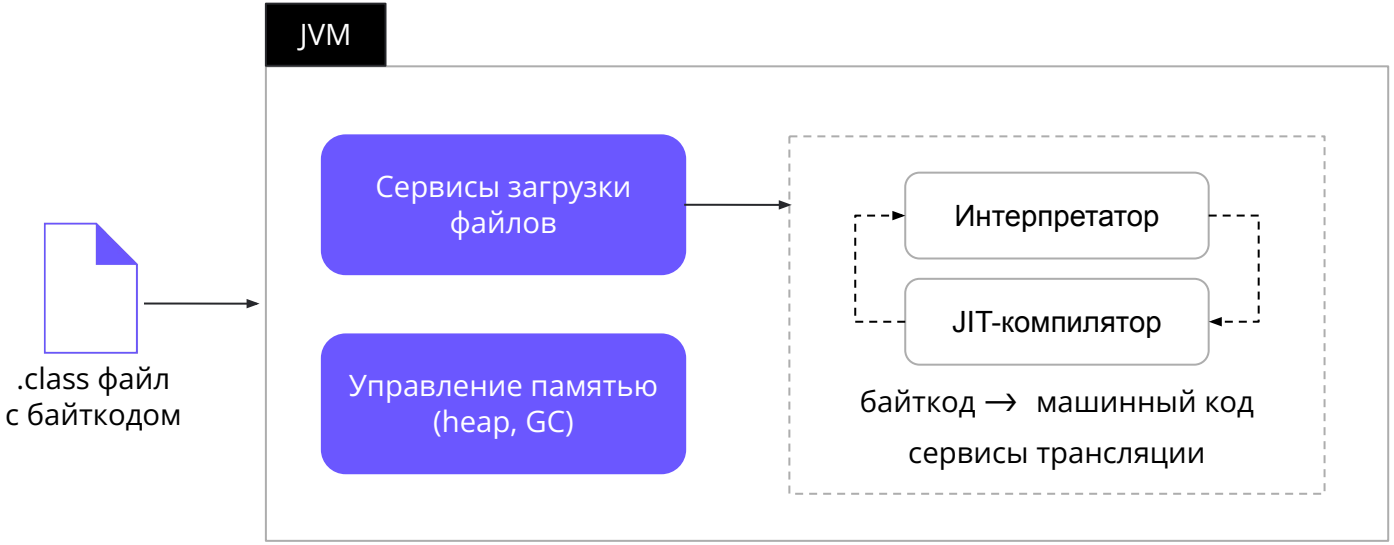


Java байткод

```java
public class Main {
    public static void main(String[] args) {
        System.out.print("Hello, World!");
    }
}
```

```
public class examples/Main {

  public <init>()V
   L0
    LINENUMBER 3 L0
    ALOAD 0
    INVOKESPECIAL java/lang/Object.<init> ()V
    RETURN
   L1
    LOCALVARIABLE this Lorg/examples/Main; L0 L1 0
    MAXSTACK = 1
    MAXLOCALS = 1

  public static main([Ljava/lang/String;)V
   L0
    LINENUMBER 5 L0
    GETSTATIC java/lang/System.out : Ljava/io/PrintStream;
    LDC "Hello, World!"
    ICONST_0
    ANEWARRAY java/lang/Object
    INVOKEVIRTUAL java/io/PrintStream.print
           (Ljava/lang/String;[Ljava/lang/Object;)Ljava/io/PrintStream;
    POP
   L1
    LINENUMBER 6 L1
    RETURN
   L2
    LOCALVARIABLE args [Ljava/lang/String; L0 L2 0
    MAXSTACK = 3
    MAXLOCALS = 1
}
```

JVM под капотом



Организация памяти

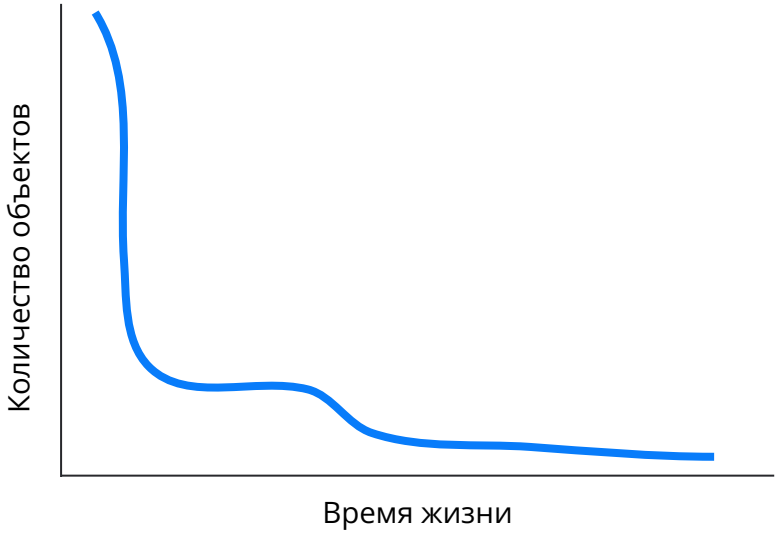
Память в JVM разделена на две части:

- Статическая память, или non-heap - создаётся во время запуска JVM. Содержит структуры классов, полей, методов и код для методов и конструкторов.
    - Загруженные классы
    - Стеки всех потоков (threads)
    - Служебная память для самой JVM
    - Маленькая часть – 128 Мб и примерно 1 Мб на каждый стек в 64-разрядной архитектуре
- Куча (Heap) - динамическая область данных, которая доступна всем JVM потокам (threads). Используется для выделения памяти для всех Java объектов.
    - Все объекты созданные во время исполнения программы
    - Большая – от нескольких MB до нескольких TB

Гипотеза слабых поколений

Объекты умирают молодыми.

Связанное утверждение: объект живущий дольше вероятно будет жить ещё дольше.



Поколения

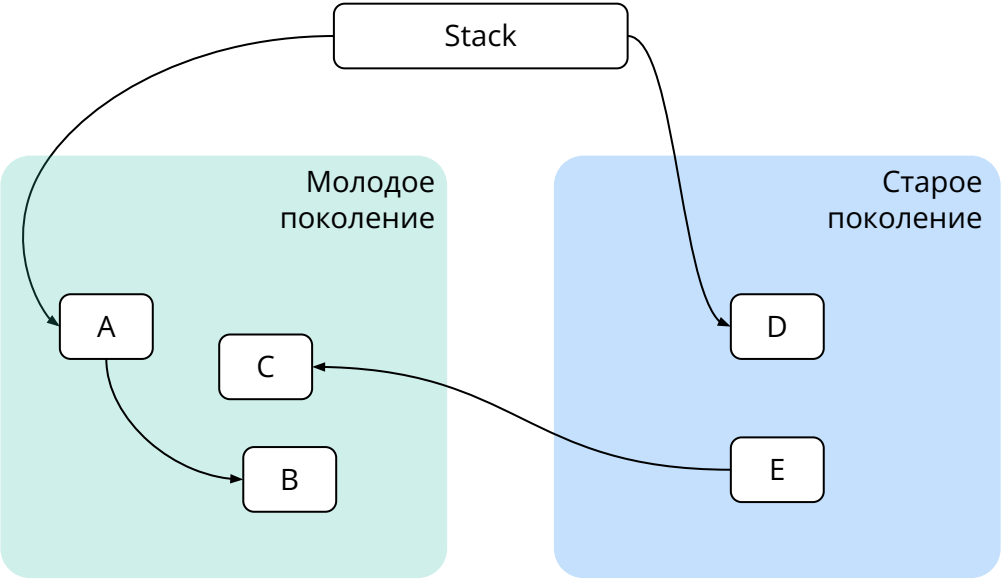
Память программы делится на два поколения:

- Молодое (Young)
- Старое (Old)

Сборщик мусора (GC) может быть поделён подобным образом:

- Частичная сборка (Small GC) собирает только молодое поколение
- Полная сборка (Full GC) собирает оба поколения

Мёртвые объекты

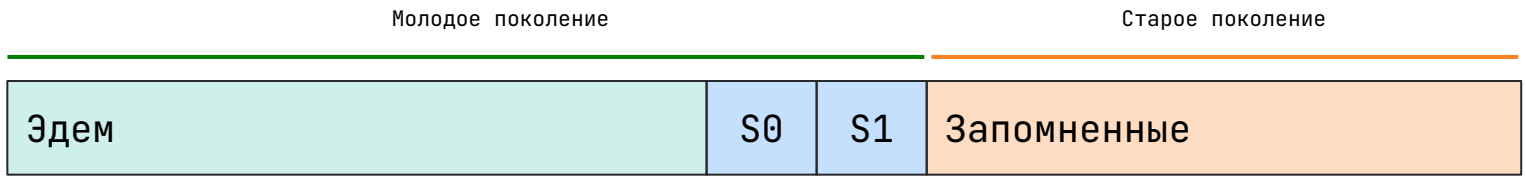


Последовательный сборщик мусора

Первым разработанным сборщиком мусора был серийный сборщик мусора. Он однопоточный. Параллельные сборщики parallel и CMS основываются на них.

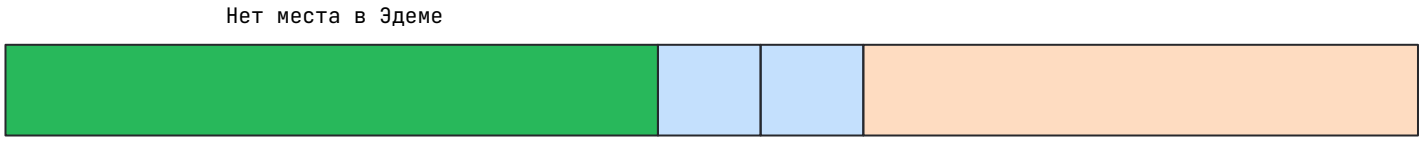
- В последовательном сборщике мусора, этот хип разделён на 4 зоны:
  - Эдем – ≈ 8/10 молодого поколения
  - Выжившие (Survivor) 0 – ≈ 1/10 молодого поколения
  - Выжившие (Survivor) 1 – ≈ 1/10 молодого поколения
  - Запомненные – ≈ 2/3 хипа

- (Почти) все объекты создаются в этой зоне Эдема.

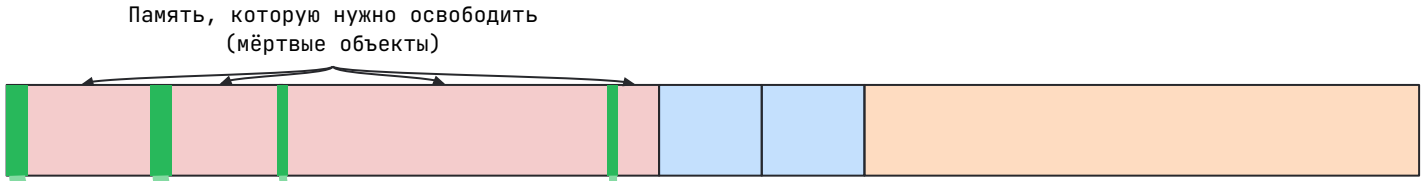


Как работает сборщик мусора

Текущее состояние:



Перед GC:

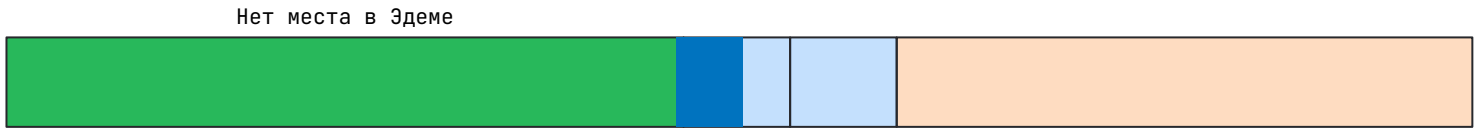


Выжившие объекты

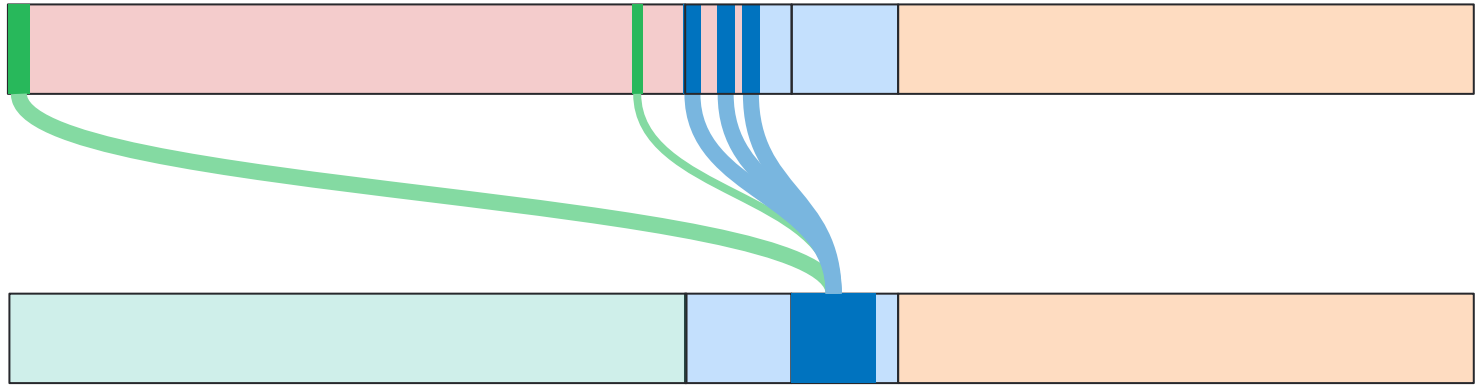
После GC:

Как работает сборщик мусора

Текущее состояние:



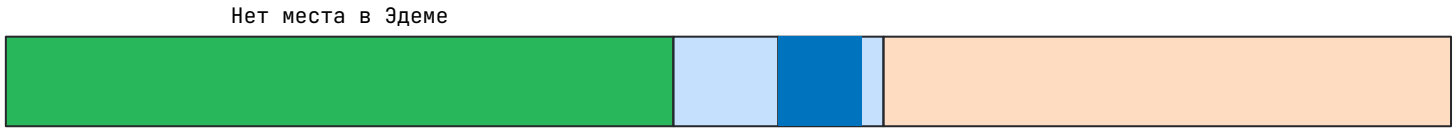
Перед GC:



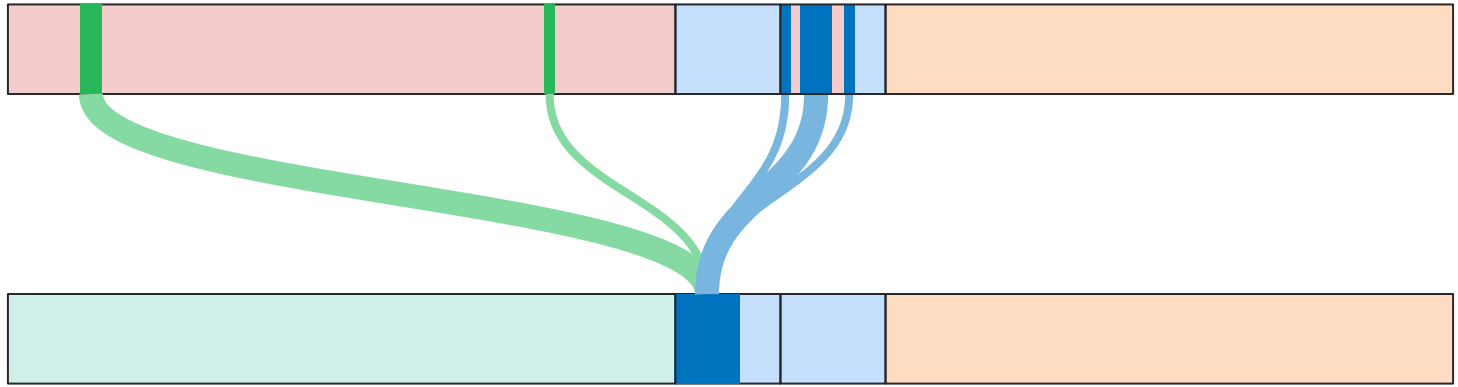
После GC:

Как работает сборщик мусора

Текущее состояние:



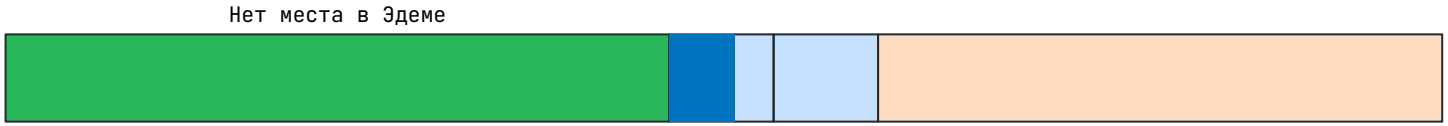
Перед GC:



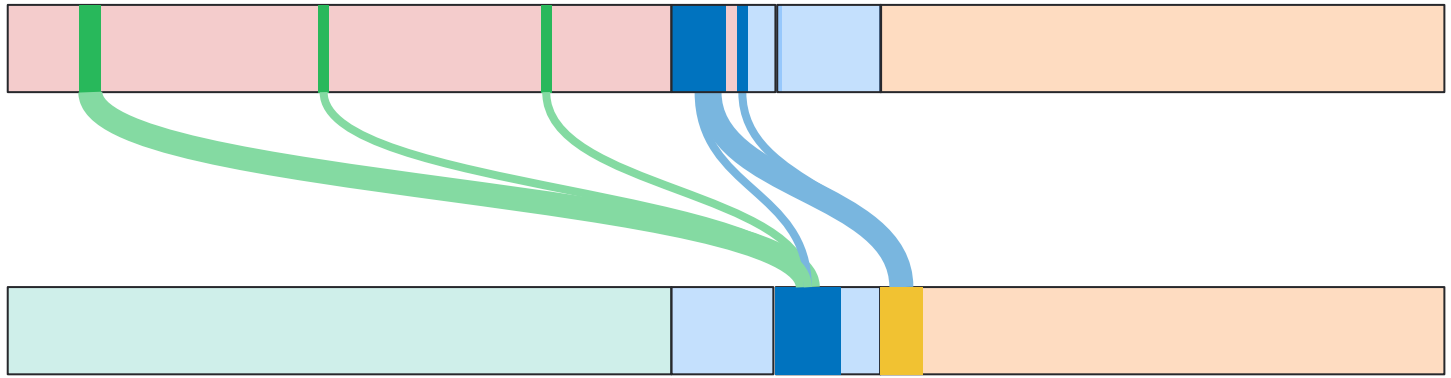
После GC:

Как работает сборщик мусора

Текущее состояние:



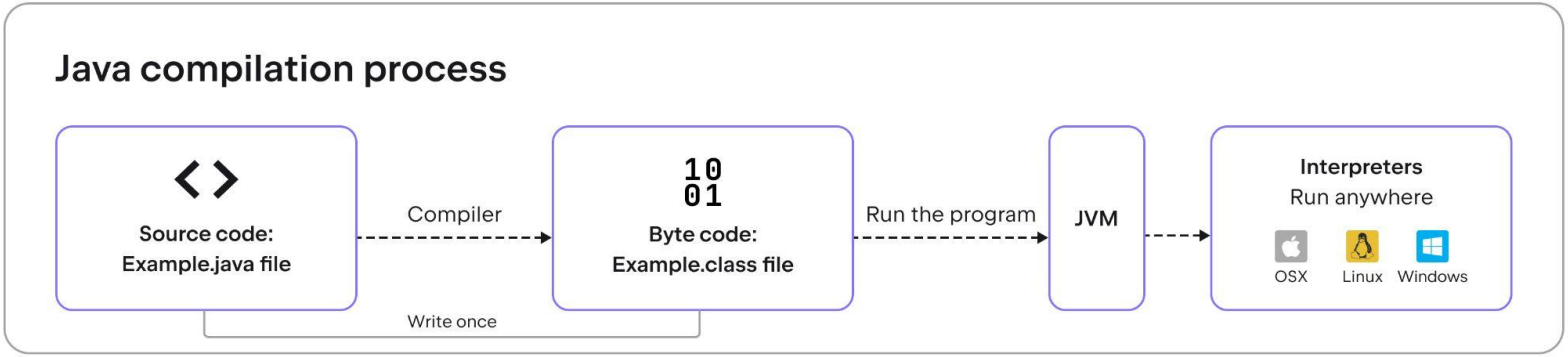
Перед GC:



После GC:

Нет свободного места перенести объекты из Эдема в Выжившие 1

Just-in-time компиляция



- Программа профилируется в рантайме.
- Для оптимизации времени выполнения - кусочки кода компилируются нативно.

Интерпретирование программ намного медленнее их исполнения непосредственно на процессоре.

Зачем нам тогда вообще интерпретатор?

Just-in-time компиляция

Для чего нам тогда вообще интерпретатор?

Интерпретатор

- Запускается мгновенно.
- Производительность выполняемого кода - довольно низкая.

VS

JIT-компилятор

- Стартует спустя долгое ожидание (нужно время для оптимизаций).
- Производительность исполняемого (скомпилированного) кода высока.

Just-in-time compilation

Какой код стоит JIT компиляции?

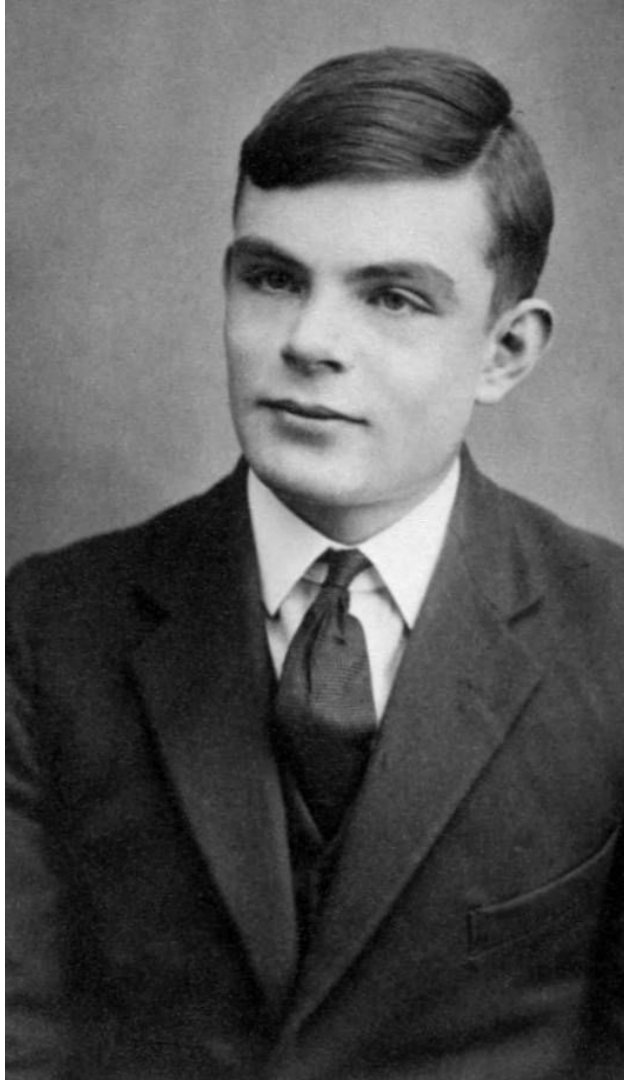
Код, который выполняется долго или код, который выполняется очень часто, из-за чего оверхед из-за компиляции будет покрыт профитом от выполнения оптимизированного кода.

Just-in-time compilation

Как же понять, какие куски кода будут выполняться долго?

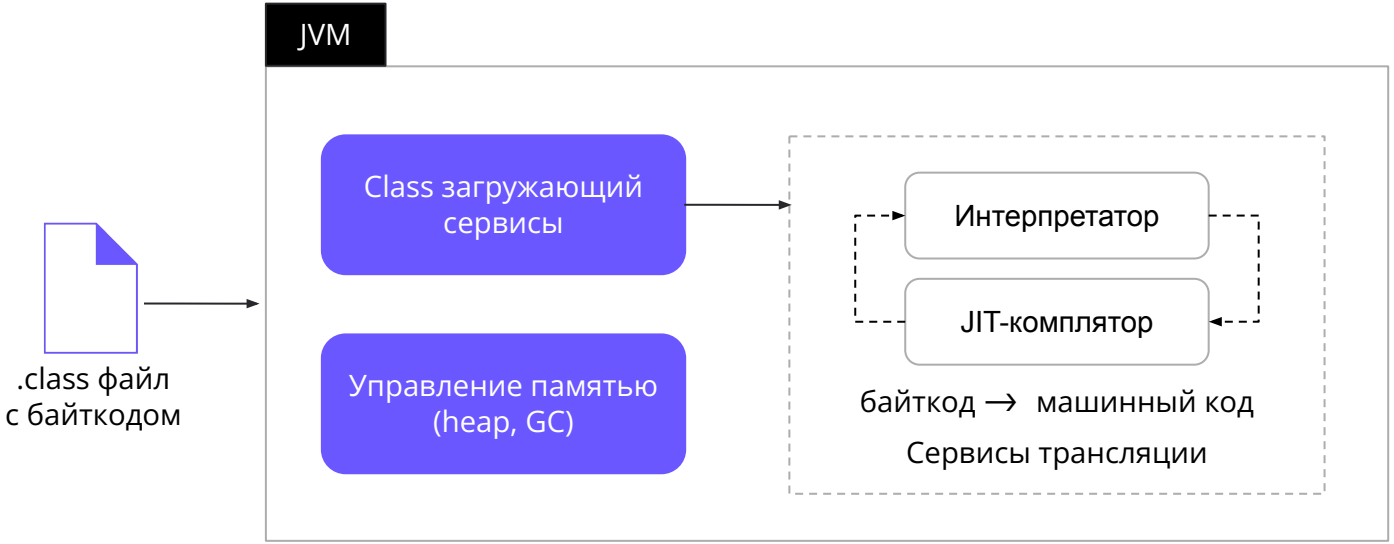
Один парень по имени Алан Тьюринг сказал, что это **невозможно**.

Но, имперически, это возможно. Если кусок кода выполняется долго хотя бы раз, то очень вероятно, что и он же будет выполняться так долго и в следующий раз.



JVM под капотом

Для чего компилятору нужен доступ к интерпретатору?



JVM под капотом

Для чего компилятору нужен доступ к интерпретатору?

```
class PiUtils {
    private static final double PI = 3.141592653589;

    public static double getPiSquared() {
        return PI * PI;
    }
}
```

```
public static double getPiSquared() {
    return 9.869604401084375;
}
```

JVM под капотом

Для чего компилятору нужен доступ к интерпретатору?

```
class PiUtils {
    private static final double PI = 4;

    public static double getPiSquared() {
        return PI * PI;
    }
}
```

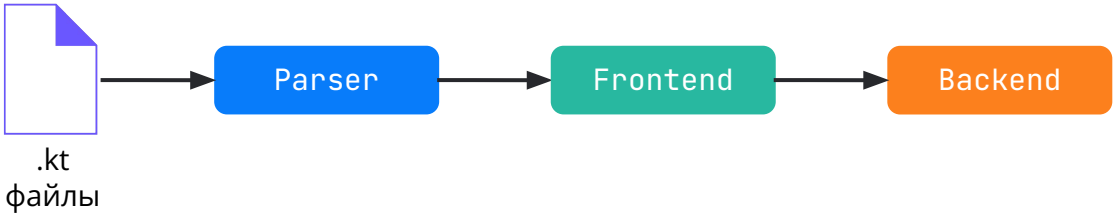
Reflection

```
public static double getPiSquared() {
    return 9.869604401084375;
}
```

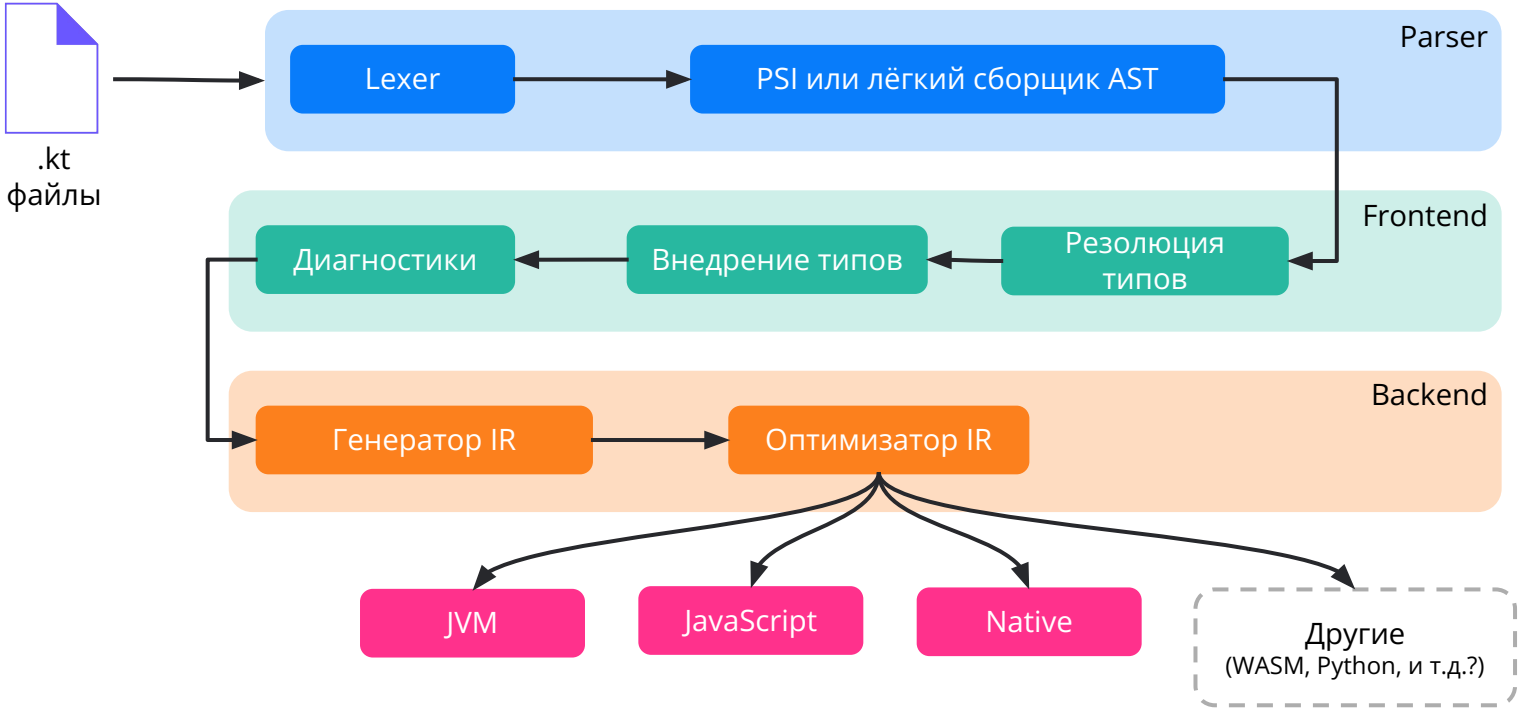
Язык Kotlin

- Может компилироваться в JVM байткод.
- Совместим с Java.
  - Может работать в Java проектах.
  - Может использовать Java библиотеки.
- Безопасный и лаконичный.
- Предоставляет возможность внедриться в процесс компиляции (compiler plugins).
- Главный язык для разработки Android-приложений (согласно Google).

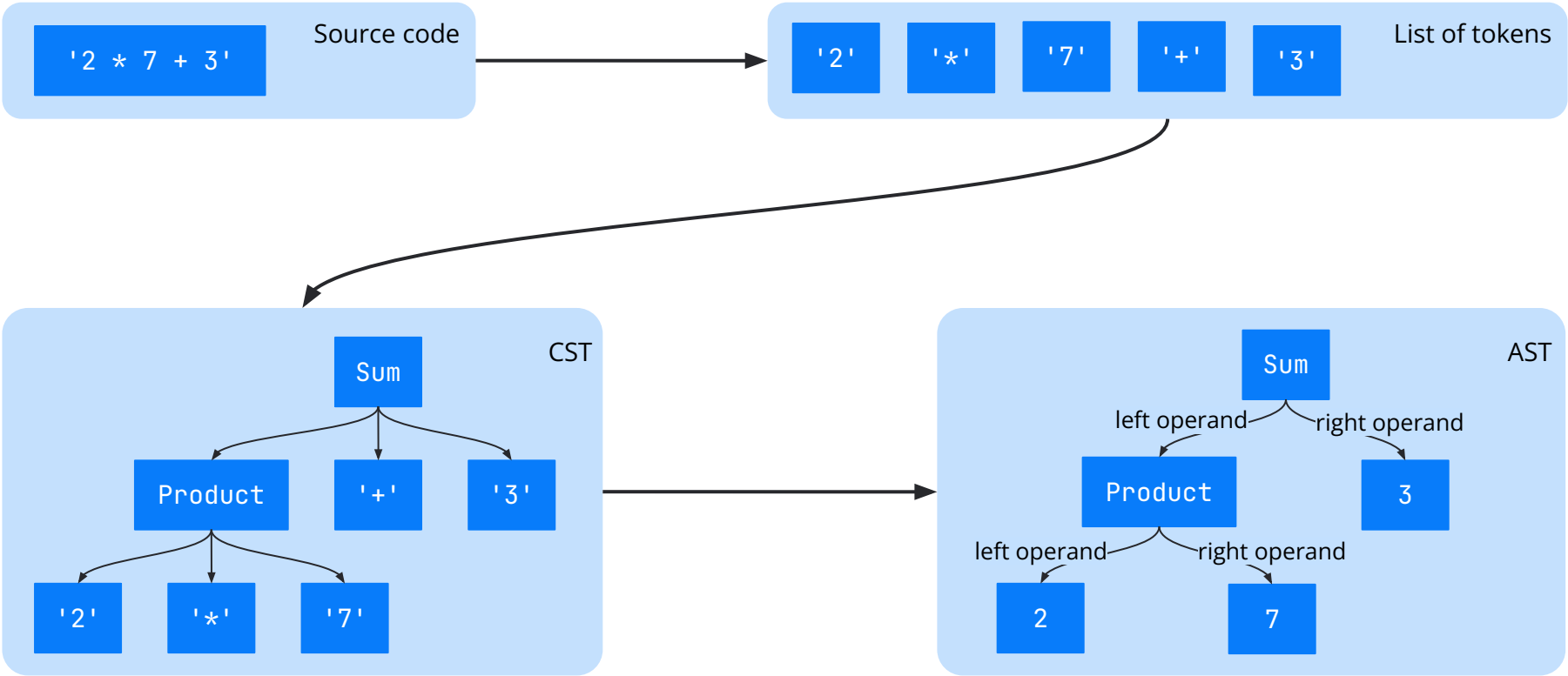
Kotlin компилятор



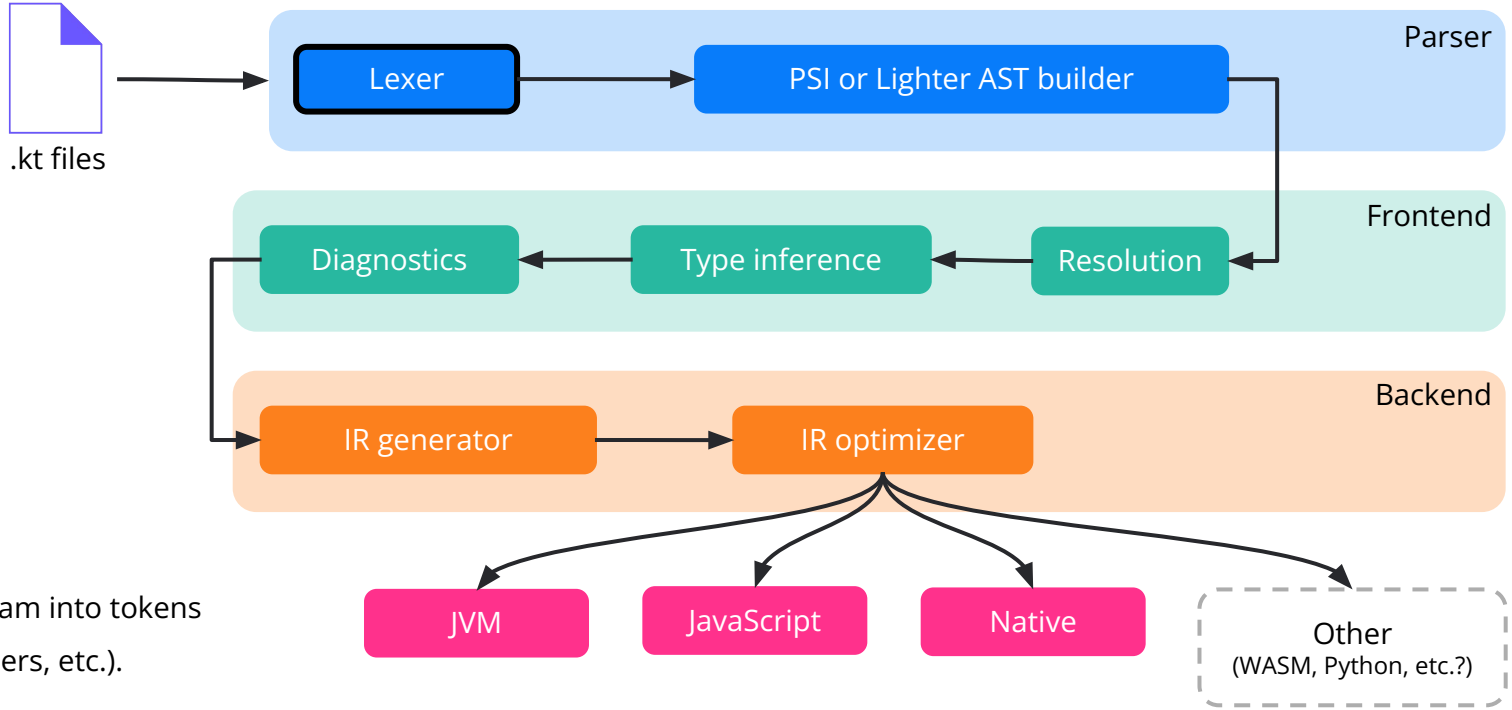
Kotlin компилятор



Parsing in a nutshell

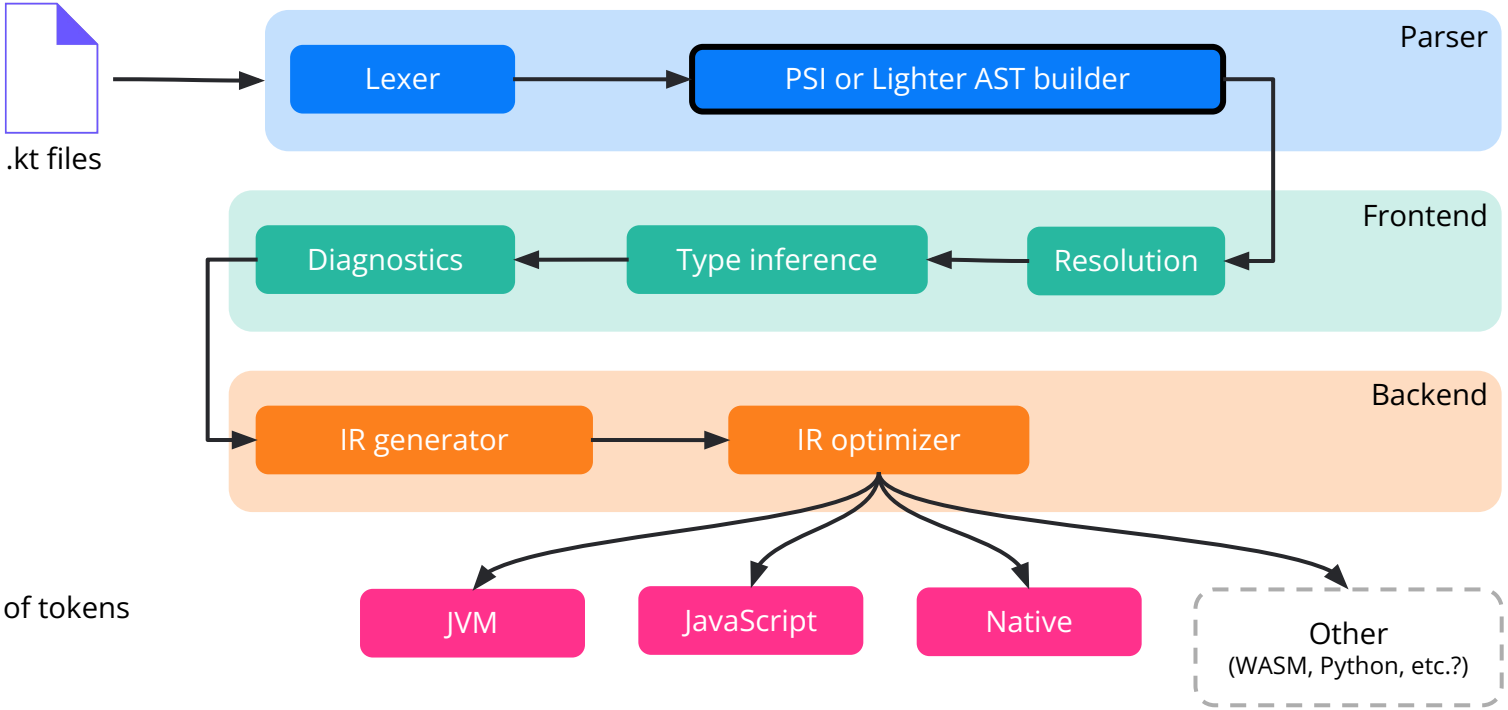


Kotlin compiler



Splitting the program into tokens (keywords, identifiers, etc.).

Kotlin compiler



Converting the list of tokens into CST or AST

Kotlin compiler: PSI (Program structure interface)

PSI Viewer: Functions.kt

Show PSI structure for: Kotlin file | Show PsiWhiteSpace | Show tree nodes | Extension: .kt

Text

```
6 }
7
8 @FirstAnnotation
9 private fun barPrivate() {
10     println("private first example bar")
11 }
12
```

PSI Tree

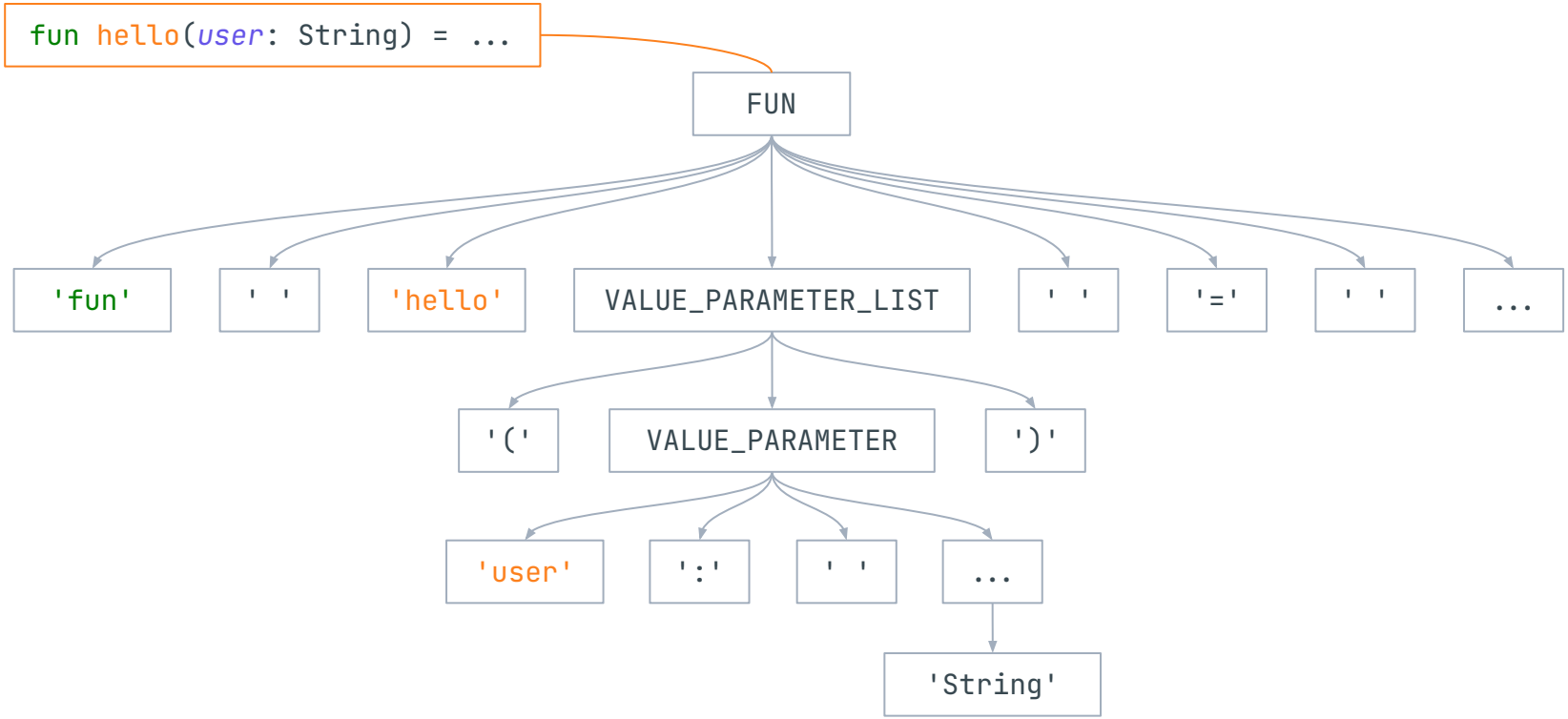
- FUN
  - MODIFIER_LIST
  - PsiWhiteSpace
  - PsiElement(fun)
  - PsiWhiteSpace
  - PsiElement(IDENTIFIER)
  - VALUE_PARAMETER_LIST
  - PsiWhiteSpace
  - BLOCK
- PsiWhiteSpace
- FUN
- PsiWhiteSpace
- CLASS
- PsiWhiteSpace
- FUN
- PsiWhiteSpace
- FUN
- PsiWhiteSpace
- FUN
- PsiWhiteSpace

References | Block Structure | Stub Structure

- KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
    - KotlinBlock Indent: CONTINUATION_WITHOUT_FIRST
    - KotlinBlock Indent: NONE
    - KotlinBlock Indent: CONTINUATION_WITHOUT_FIRST
    - KotlinBlock Indent: CONTINUATION_WITHOUT_FIRST
    - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE
  - KotlinBlock Indent: NONE

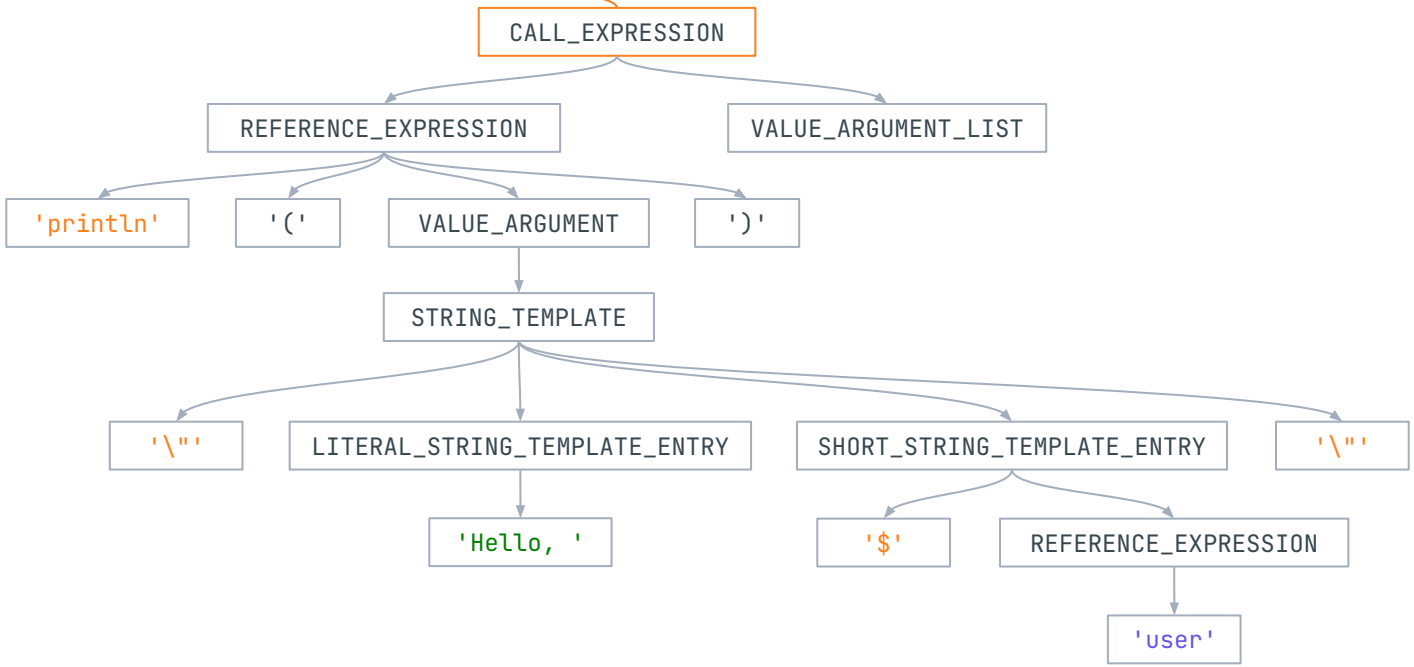
Copy PSI | Close | Build PSI Tree

Kotlin compiler: PSI



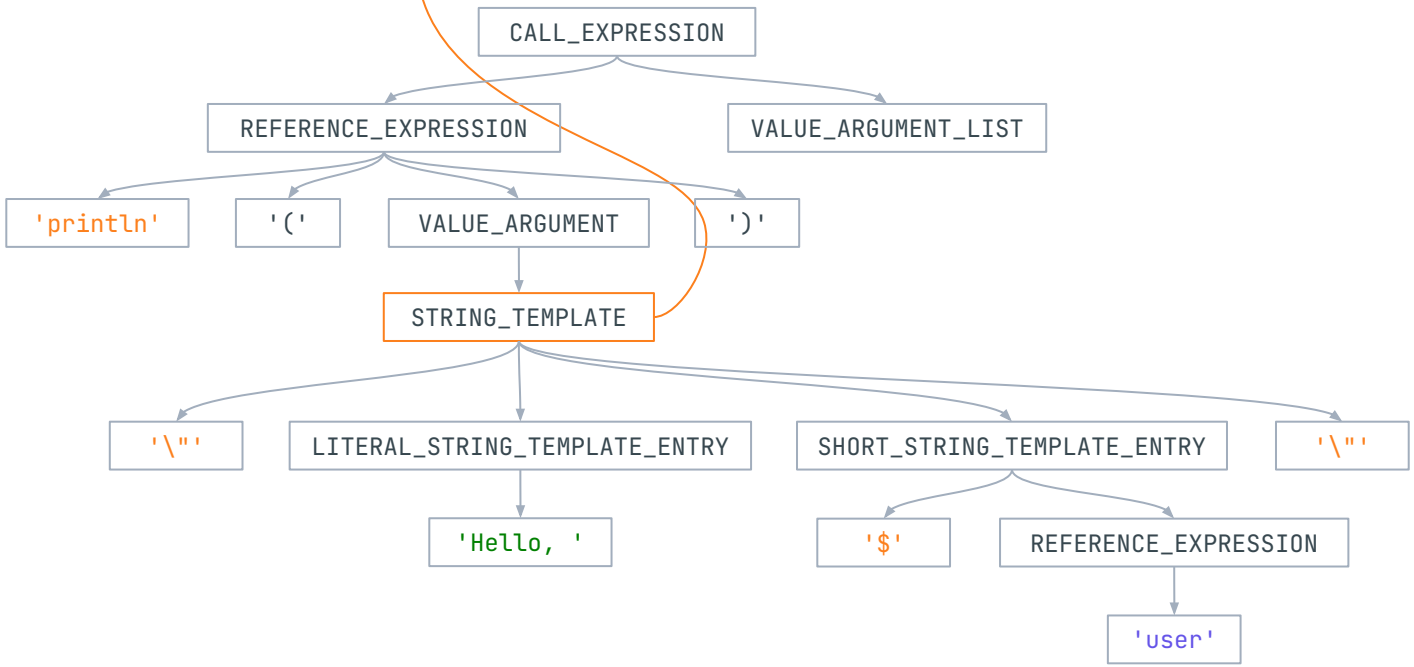
Kotlin compiler: PSI

```
fun hello(user: String) = println("Hello, $user")
```



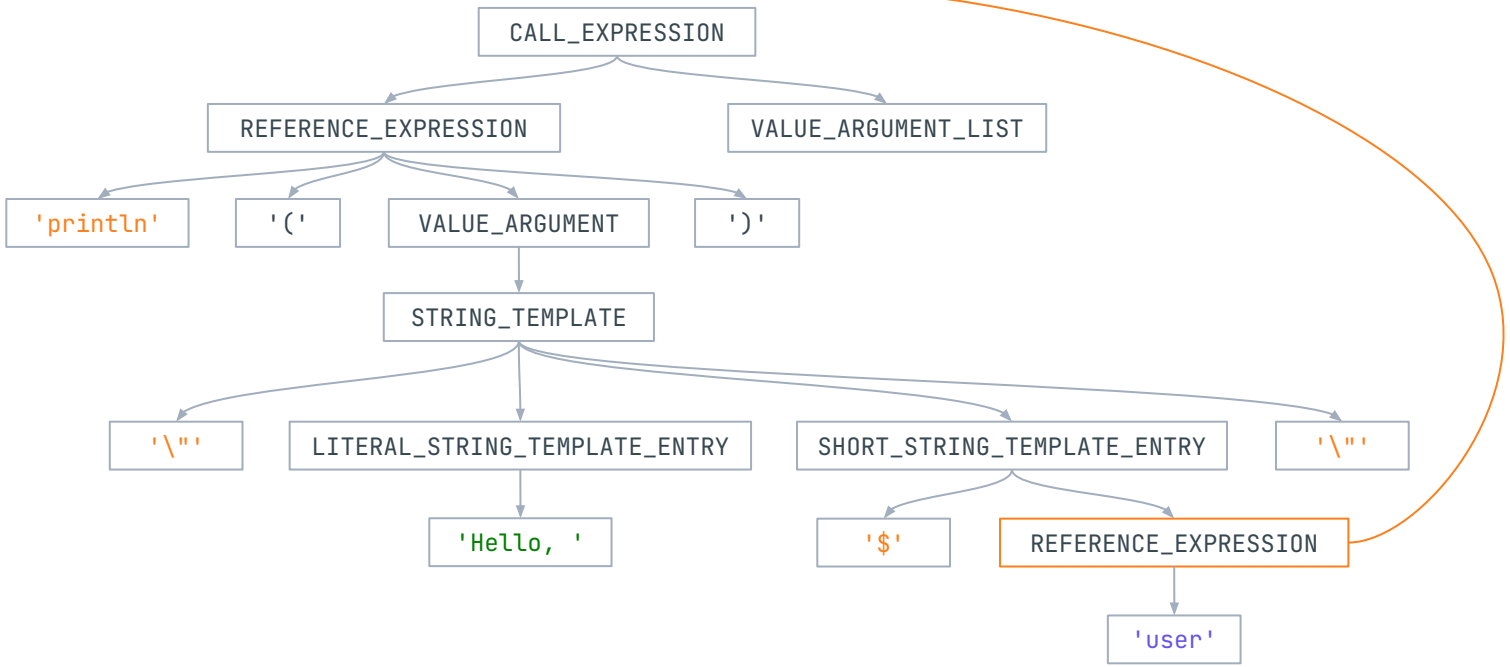
Kotlin compiler: PSI

```
fun hello(user: String) = println("Hello, $user")
```

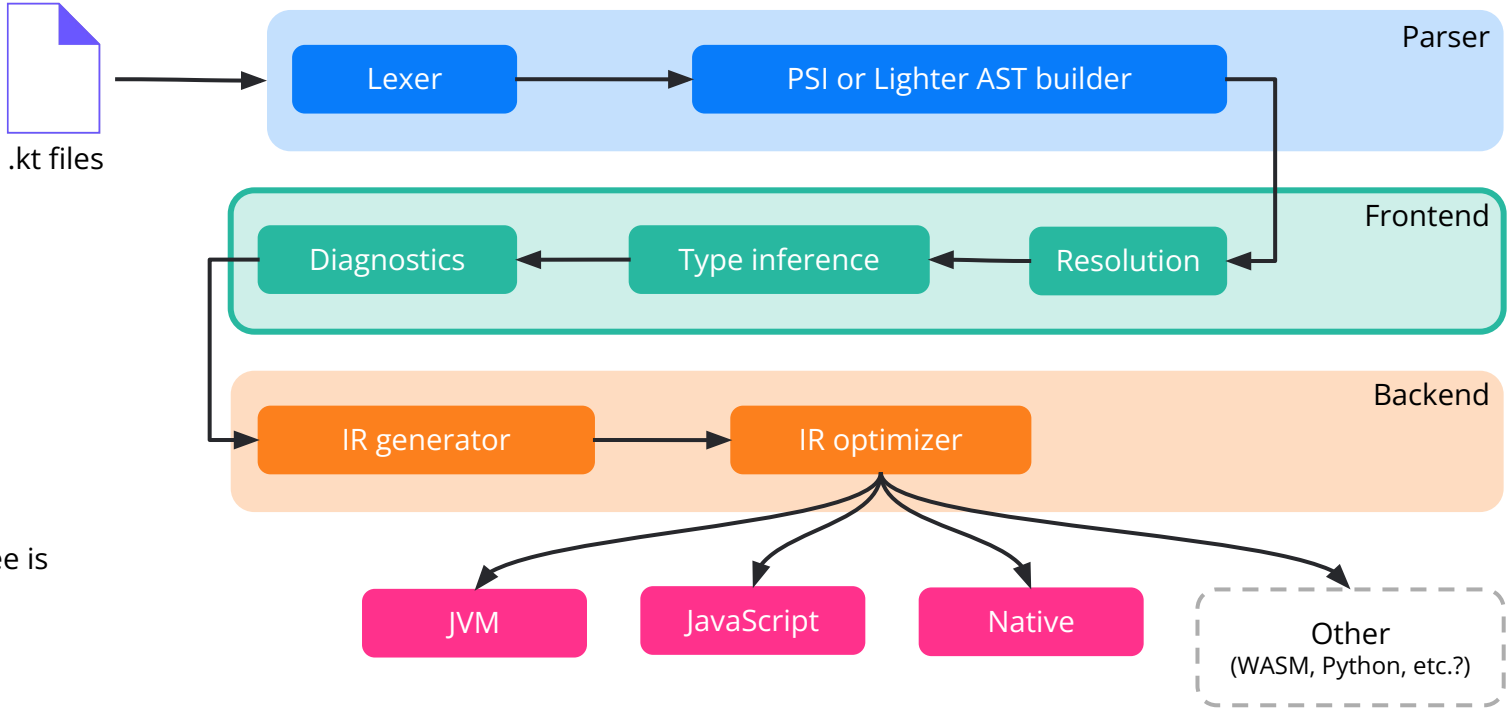


Kotlin compiler: PSI

```
fun hello(user: String) = println("Hello, $user")
```

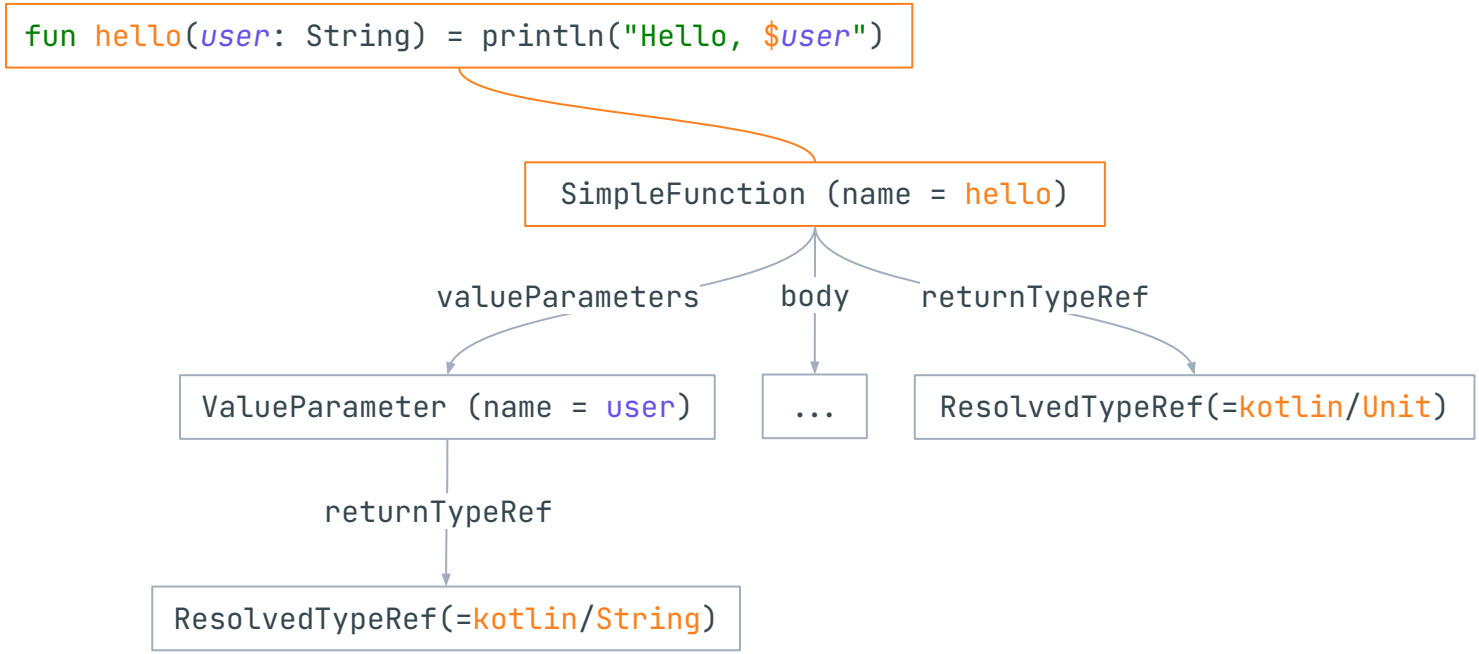


Kotlin compiler

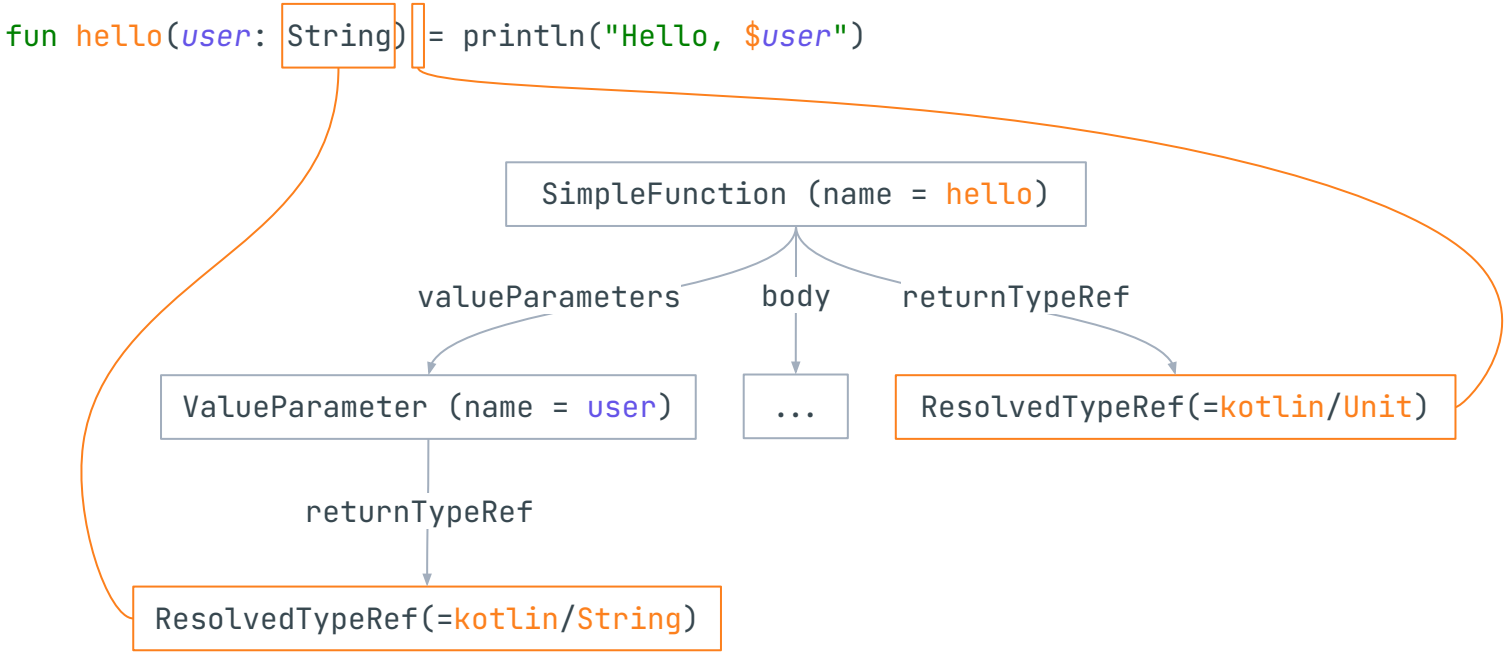


The so-called FIR tree is being built,. It's an analogue of the PSI, but it's mutable.

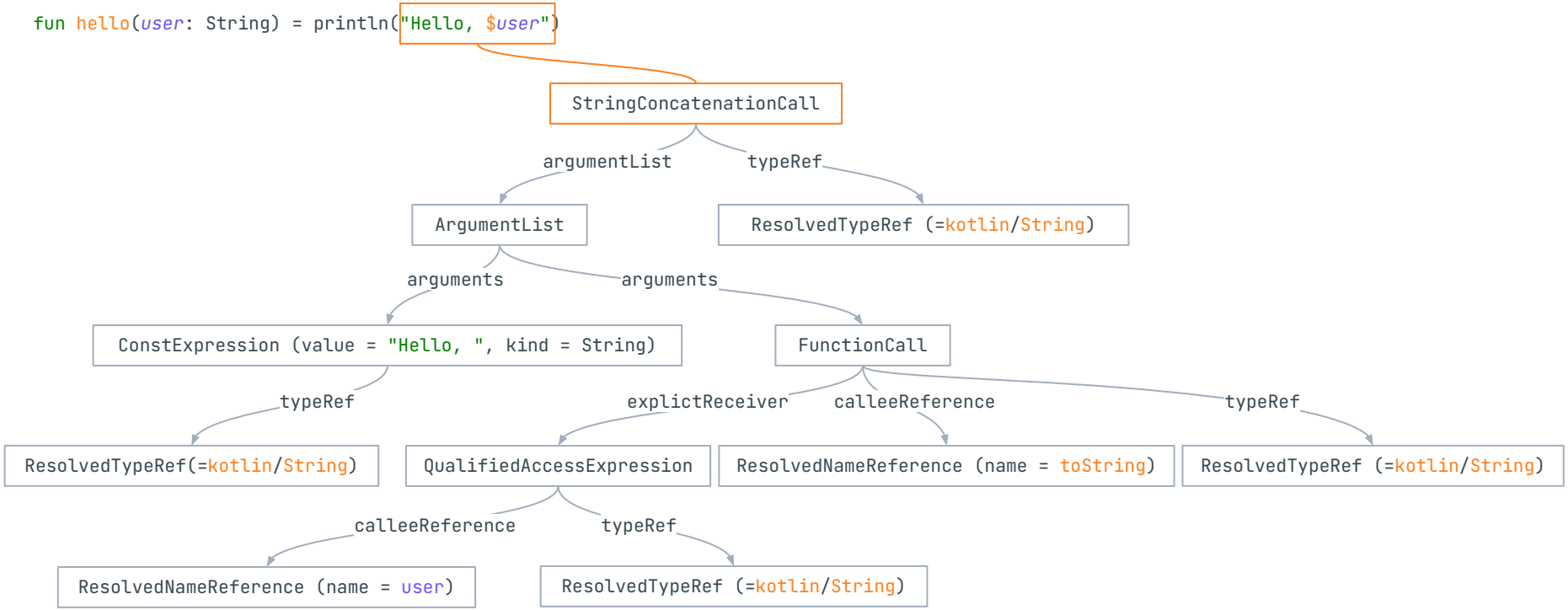
FIR? Another tree!



FIR? Another tree!

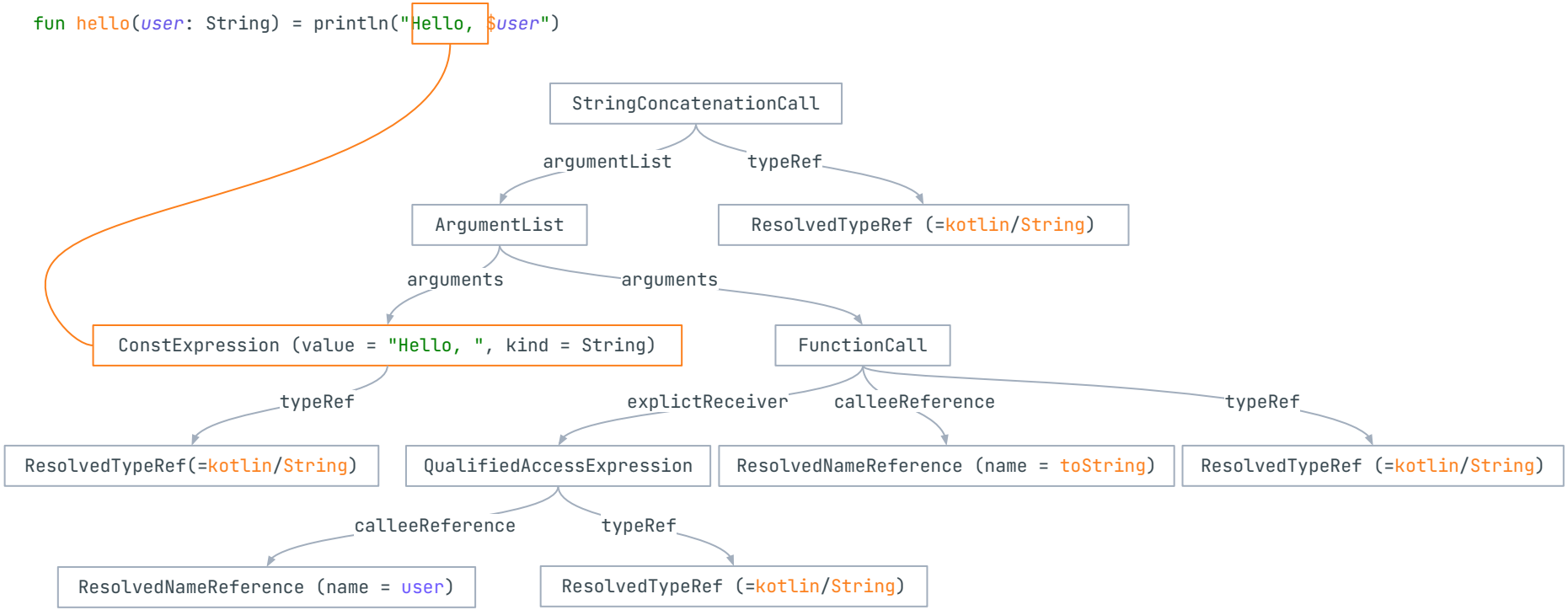


FIR? Another tree!



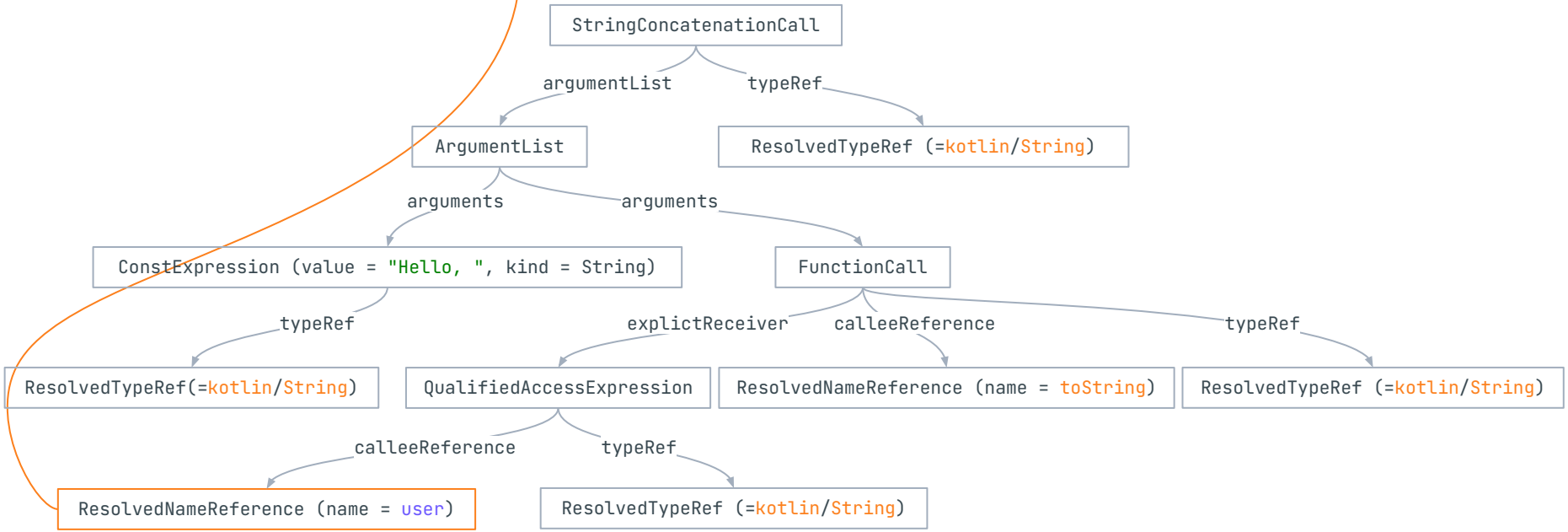
FIR? Another tree!

```
fun hello(user: String) = println("Hello, $user")
```



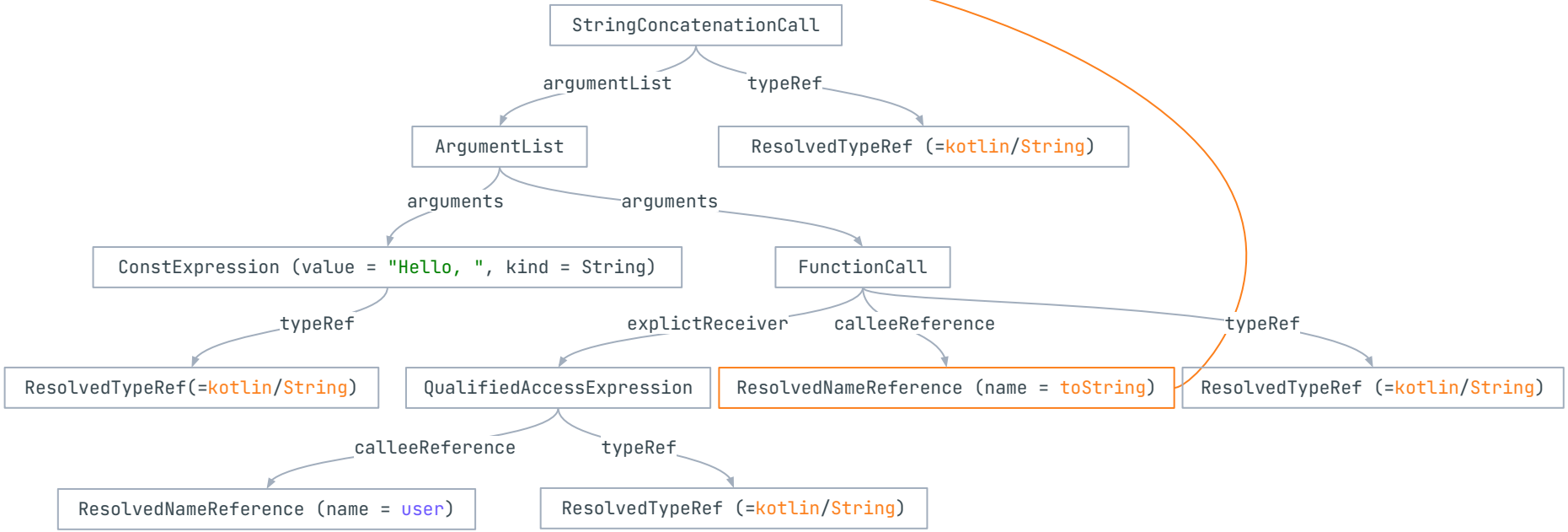
FIR? Another tree!

```
fun hello(user: String) = println("Hello, $user")
```



FIR? Another tree!

```
fun hello(user: String) = println("Hello, $user")
```



FIR: desugaring

```
if (b) {
   println("Hello")
}
```

```
when {
   b → println("Hello")
}
```

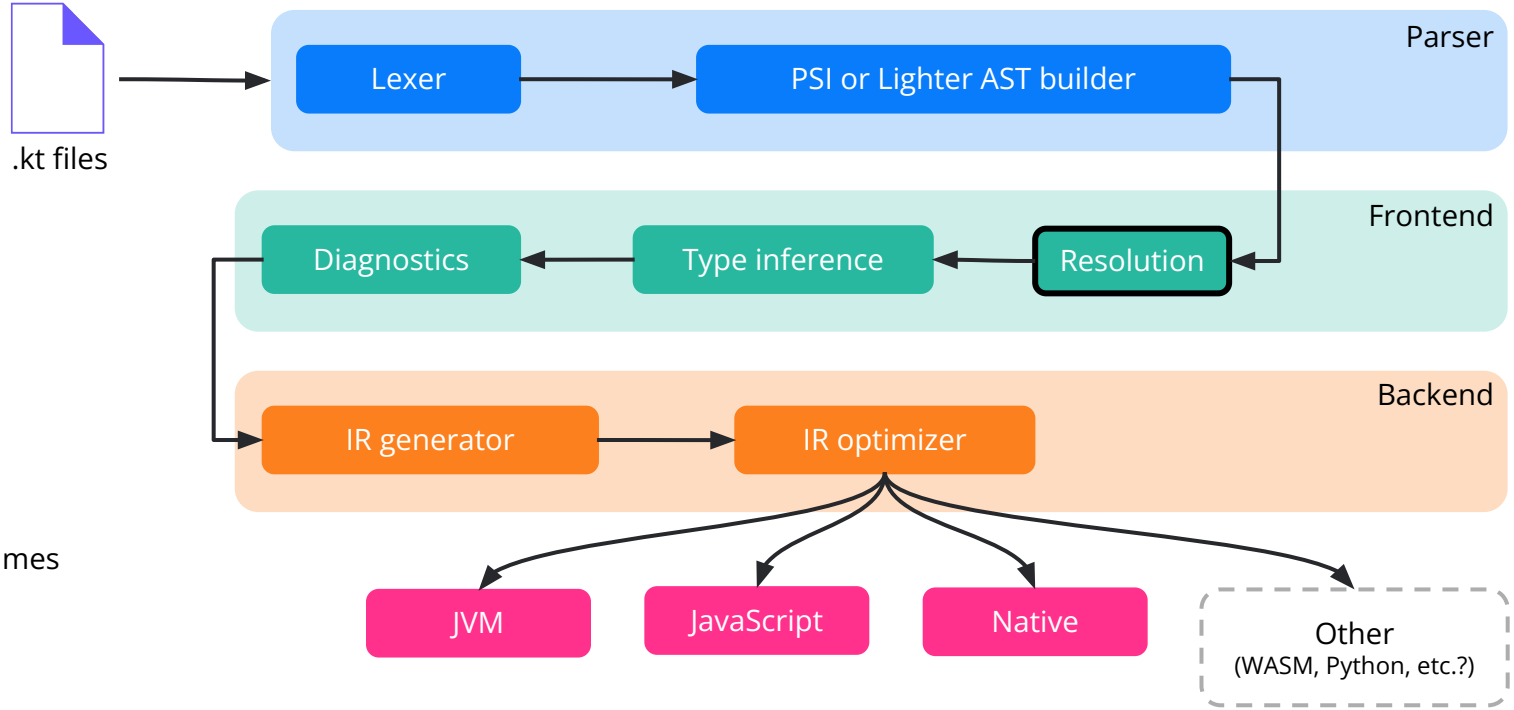
```
for (s in list) {
   println(s)
}
```

```
val <iterator> = list.iterator()
while (<iterator>.hasNext()) {
   val s = <iterator>.next()
   println(s)
}
```

```
val (a, b) = "a" to "b"
```

```
val <destruct> = "a" to "b"
val a = <destruct>.component1()
val b = <destruct>.component2()
```

Kotlin compiler



Resolving the FQ names

Kotlin compiler: resolve

```kotlin
fun myFunction() {

}
```

Library A

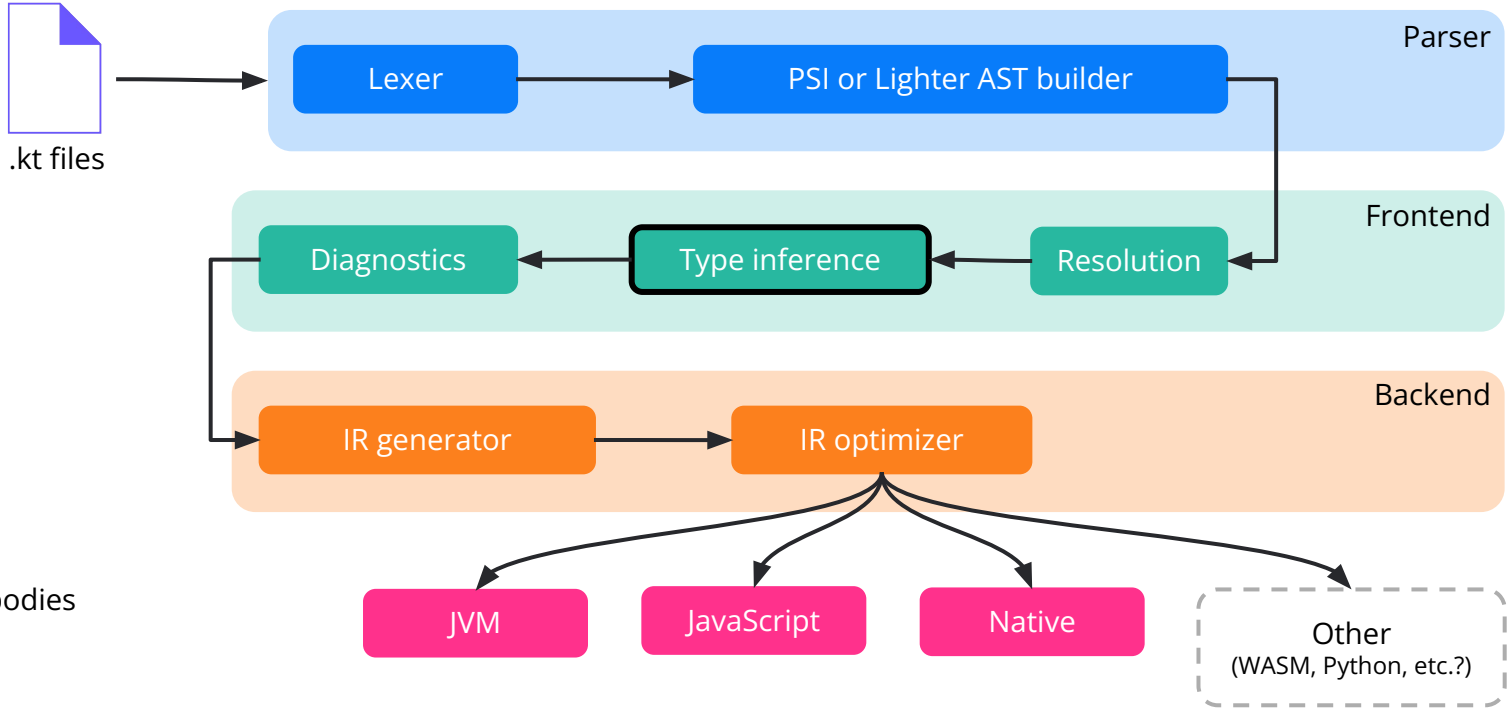
Short FQ name: `myFunction`
Resolved FQ name: `org.libraryA.myFunction`

```kotlin
fun myFunction() {

}
```

Library B

Short FQ name: `myFunction`
Resolved FQ name: `org.libraryB.myFunction`

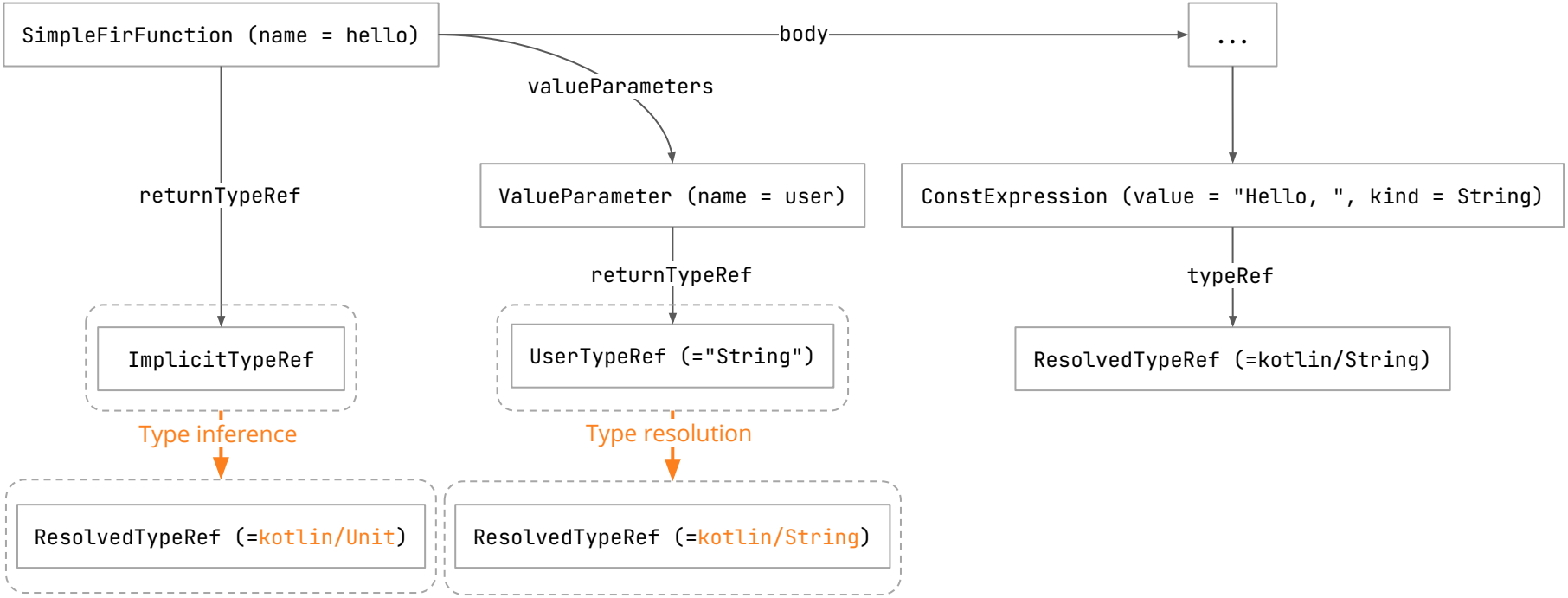
Kotlin compiler



Infers all types and
resolves functions bodies

Kotlin compiler

```
fun hello(user: String) = println("Hello, $user")
```



Java interoperability: nullability

- Java nullable types in Kotlin

Java sources

```
public class Main {
    public static String foo() {
        // TODO
    }
}
```

Kotlin sources

```
var a: ???? = foo()
```

Java interoperability: nullability

- Java nullable types in Kotlin

Java sources

```
public class Main {
    public static String foo() {
        // TODO
    }
}
```

Kotlin sources

```
var a: ???? = foo()

String!
```

Java interoperability: nullability

- Java nullable types in Kotlin
- `String!` is the type range: `[String..String?]`

Java sources

```
public class Main {
    public static String foo() {
        // TODO
    }
}
```

Kotlin sources

```
var a: ???? = foo()
```

`String!`

Java interoperability: nullability

- Java nullable types in Kotlin
- Nullability annotations `@NotNull` and `@Nullable`

Java sources

```
public class Main {
    @NotNull
    public static String foo() {
        // TODO
    }
}
```

Kotlin sources

```
var a: String = foo()
```

Java interoperability: collection mapping

- Java collection types in Kotlin

Java sources

```
public class Main {
    @NotNull
    public static List<@NotNull String> foo() {
        // TODO
    }
}
```

Kotlin sources

```
var a: ???? = foo()
```

Java interoperability: collection mapping

- Java collection types in Kotlin
- `(Mutable)List<T>` is the type range: `[MutableList<T> .. List<T>]`

Java sources

```
public class Main {
    @NotNull
    public static List<@NotNull String> foo() {
        // TODO
    }
}
```

Kotlin sources

```
var a: ???? = foo()
```

```
(Mutable)List<String>
```

Kotlin compiler: control- and data-flow analysis

- Variable initialization analysis
- Return analysis
- Smart cast analysis

```
val a: Int

while(true) {
    if (Random.nextBoolean()) {
        a = 15
        break
    }
}

println(a) // It compiles!
```

Each variable is initialized before being used.

Each immutable variable is not reassigned after initialization.

Kotlin compiler: control- and data-flow analysis

- Variable initialization analysis
- Return analysis
- Smart cast analysis

If the return type is not `Unit`, then the function won't return control unless it returns something.

```
fun bar(): Int {
    print("Again")
    while (true) {
        print(" and again")
    }
} // It compiles!

fun baz(): Long {
    error("YOLO! :)")
} // It compiles!
```

Kotlin compiler: control- and data-flow analysis

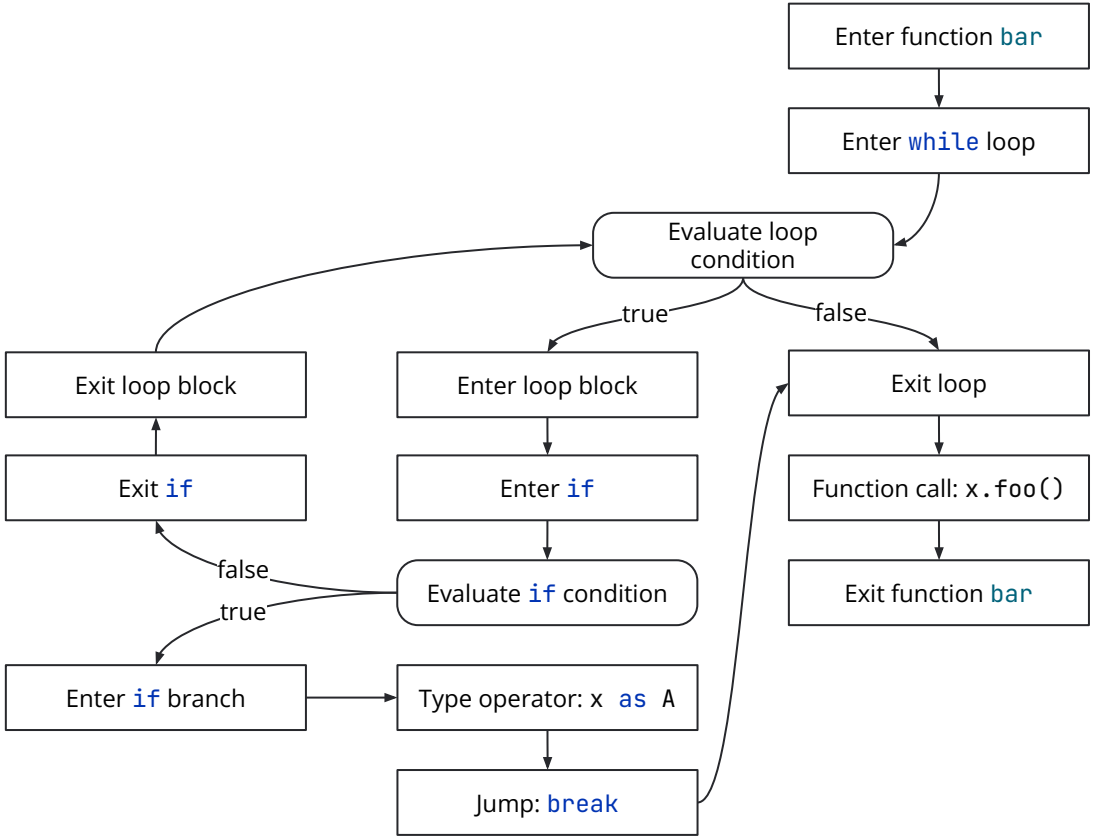
- Variable initialization analysis
- Return analysis
- Smart cast analysis

If type check is successful, then the checked value is automatically casted to the corresponding type.

```kotlin
fun Any?.printFirstElement() {
    when (this) {
        is List<*> → get(0)
        is Iterable<*> → iterator().next()
    }
}

fun String?.length(): Int =
    if (this == null) 0
    else length

fun Int?.isEven(): Boolean =
    this ≠ null && this % 2 == 0
```

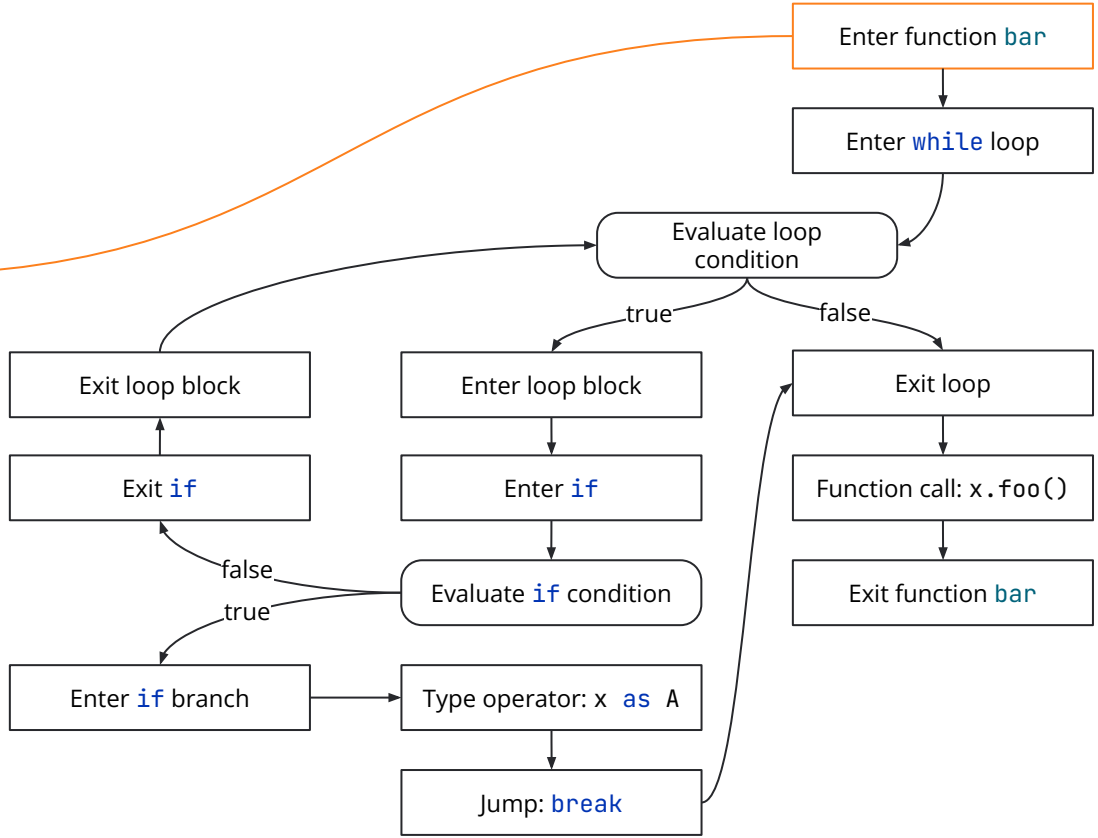
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



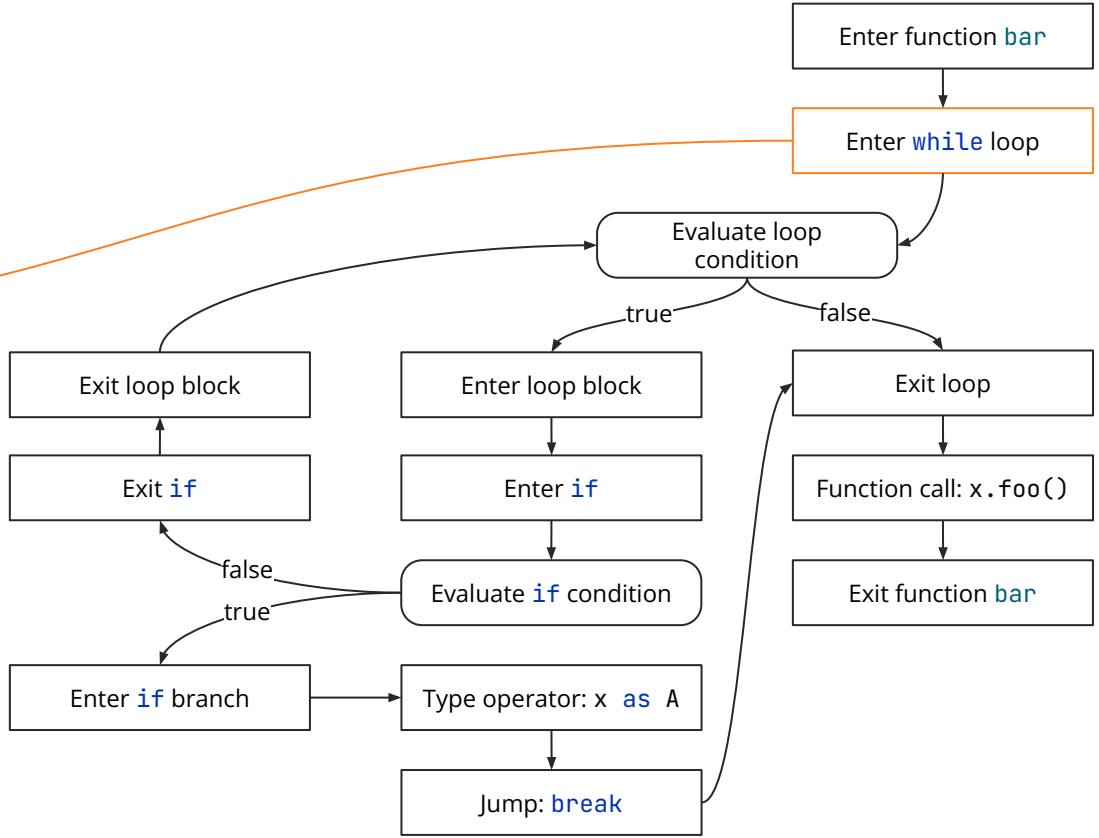
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



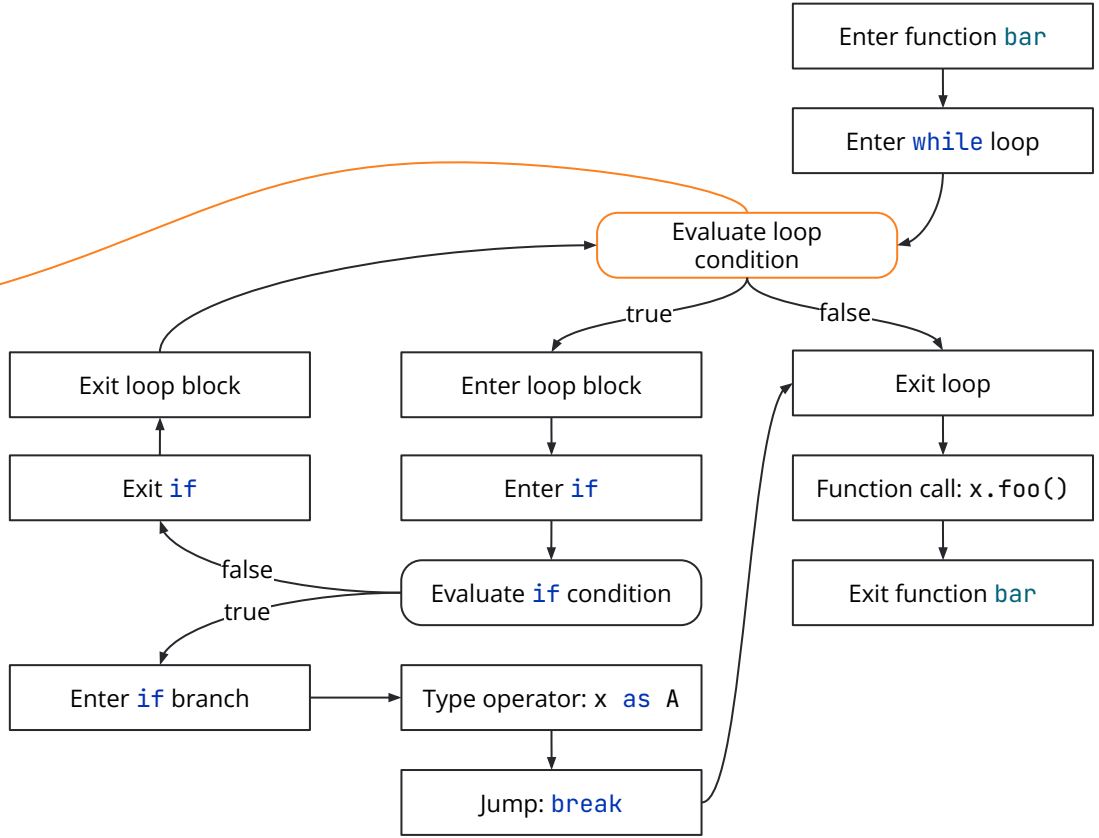
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



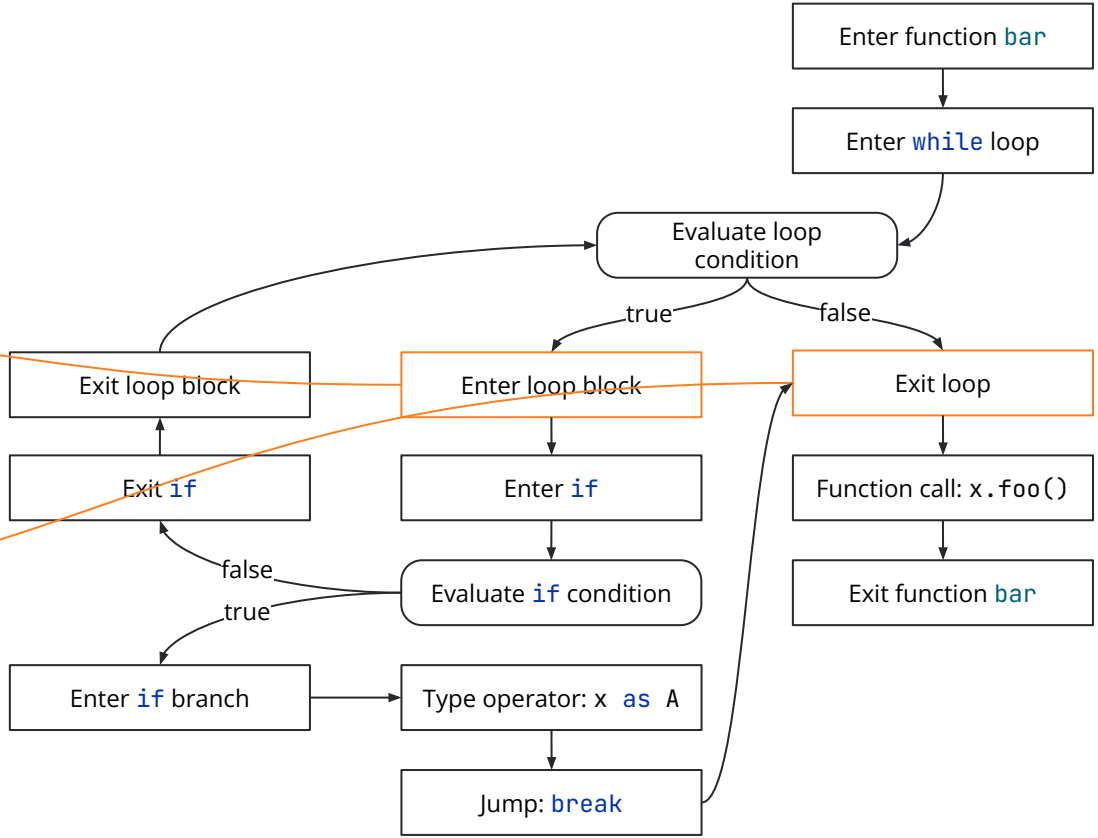
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



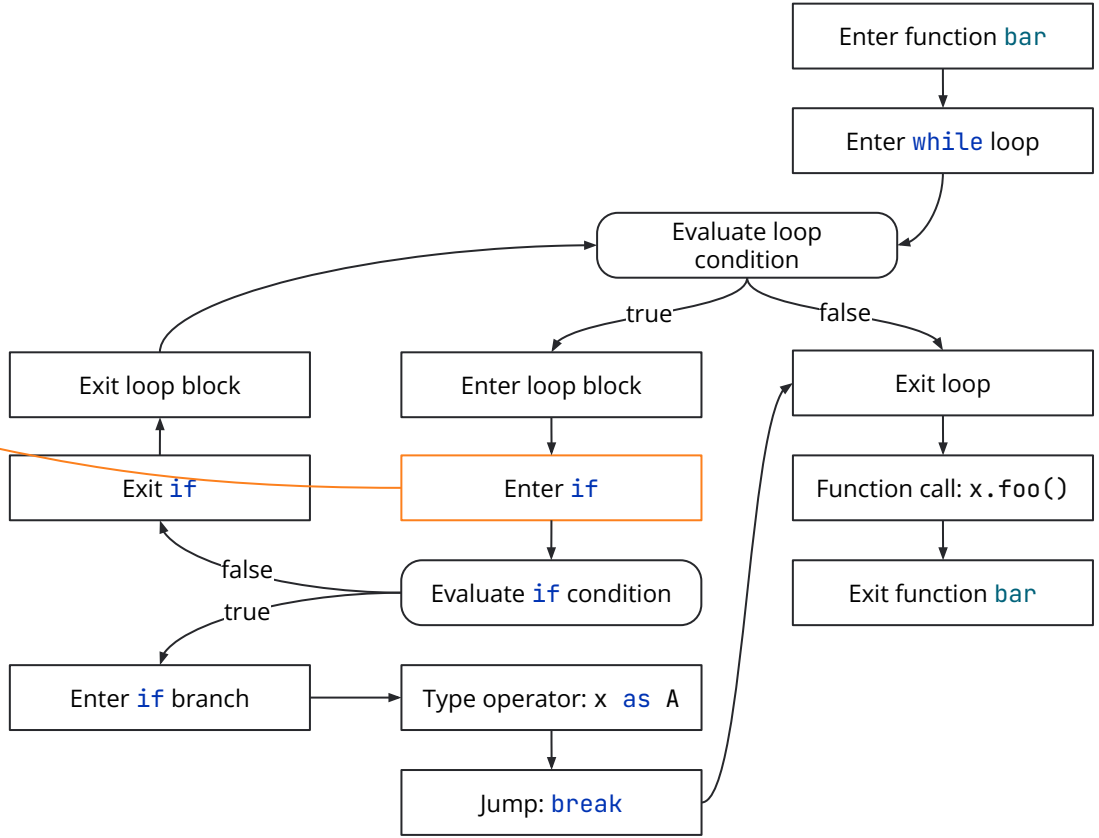
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



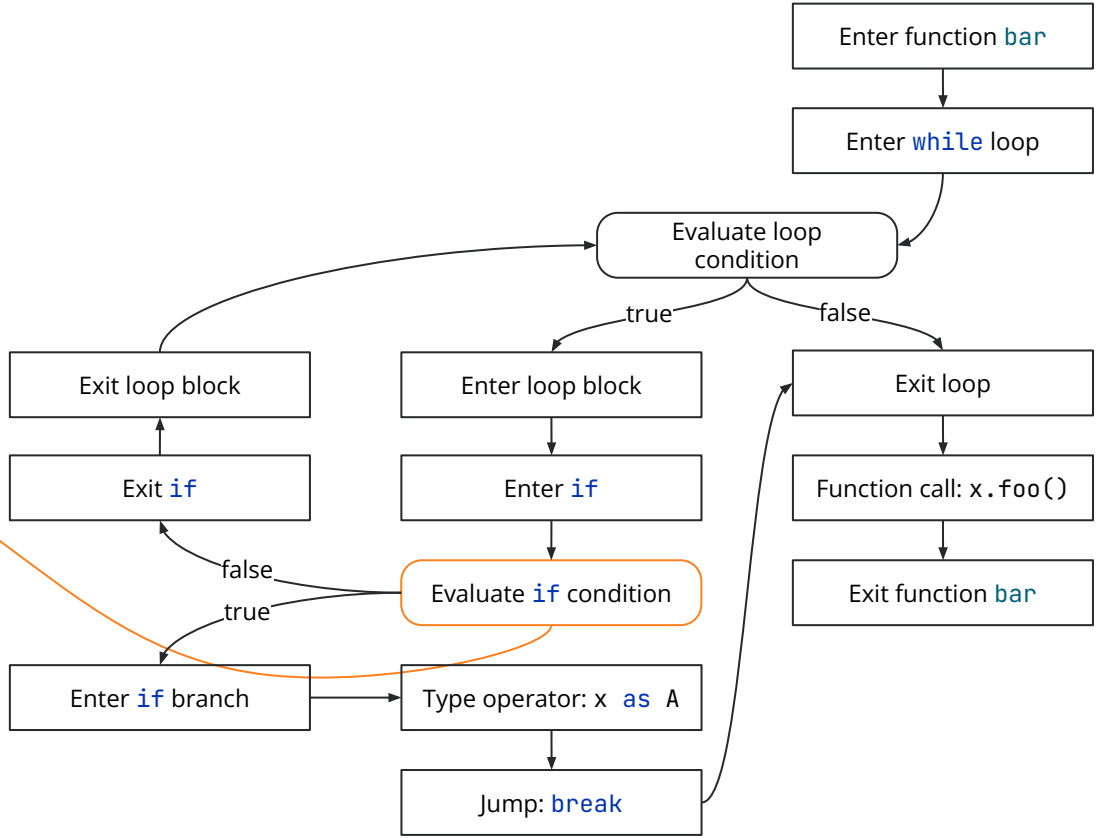
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



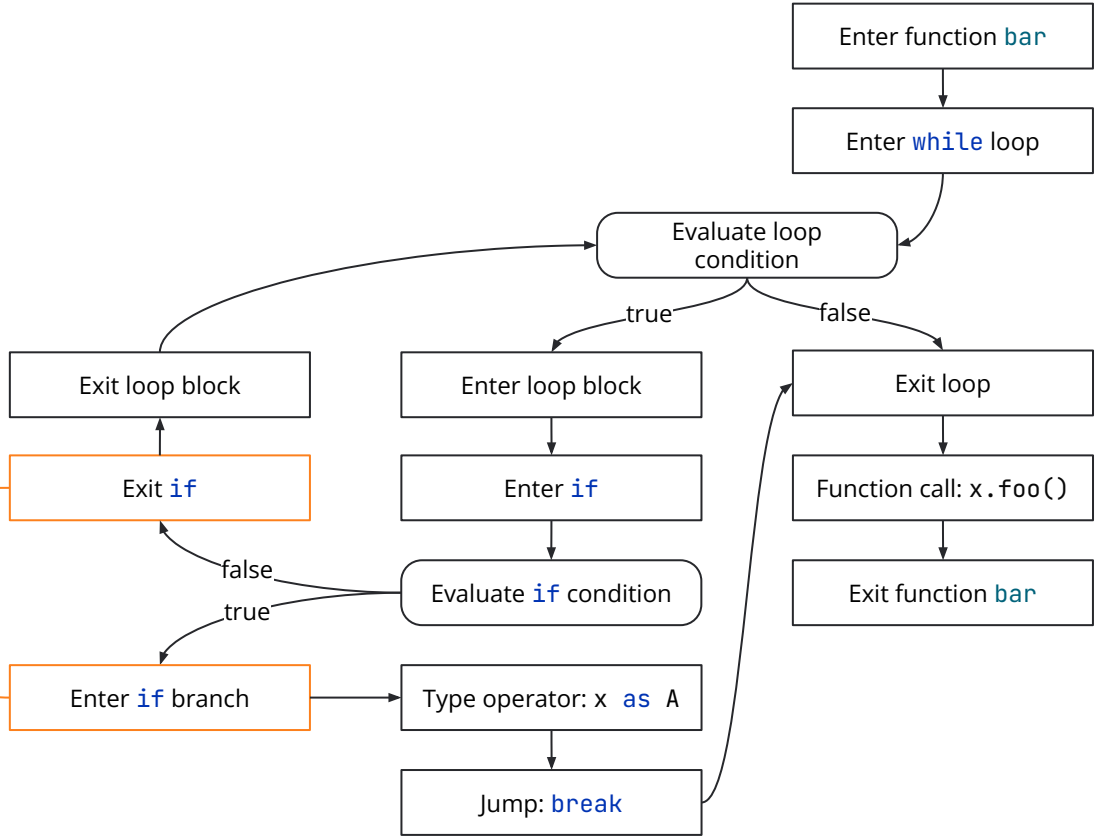
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



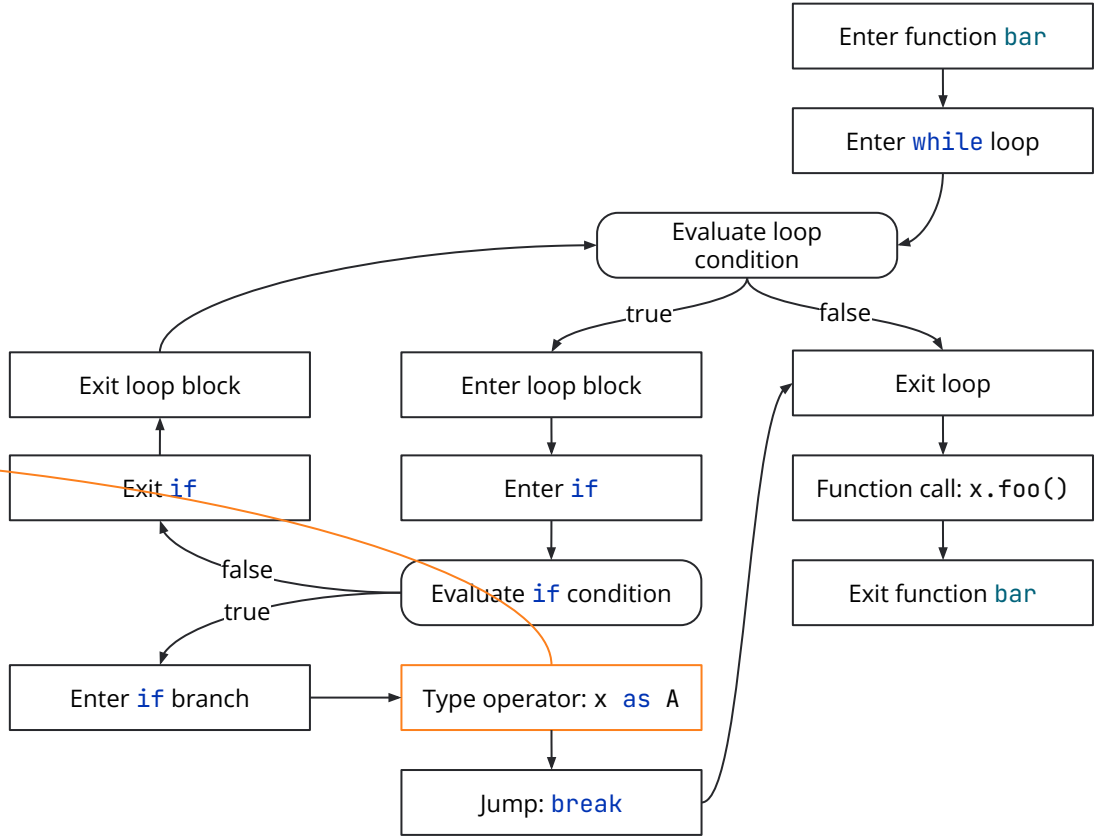
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



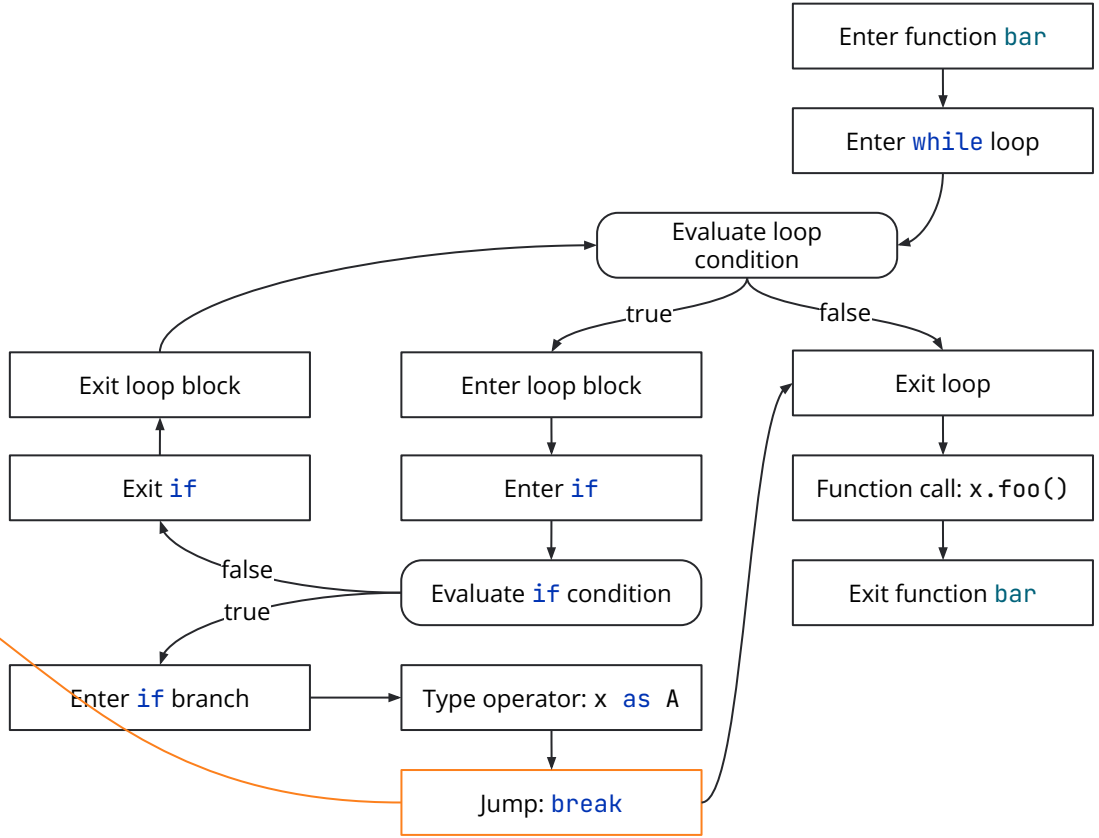
Kotlin compiler: control- and data-flow analysis

```
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



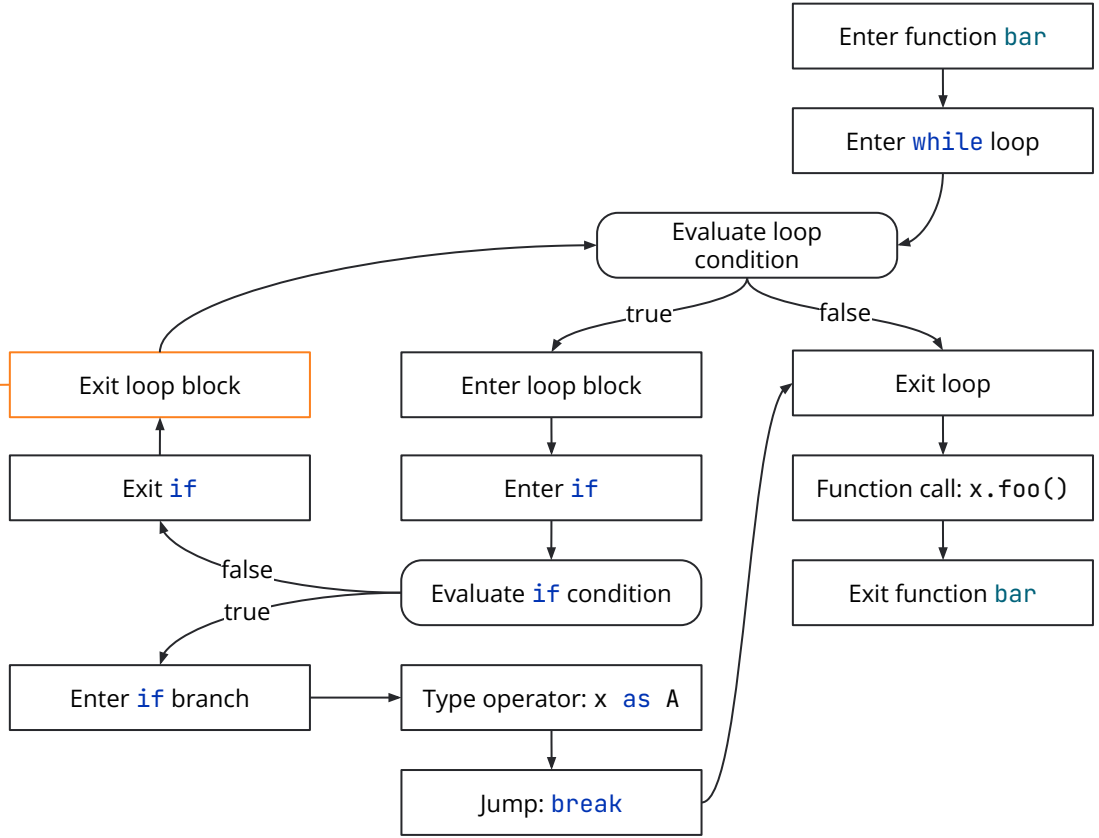
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



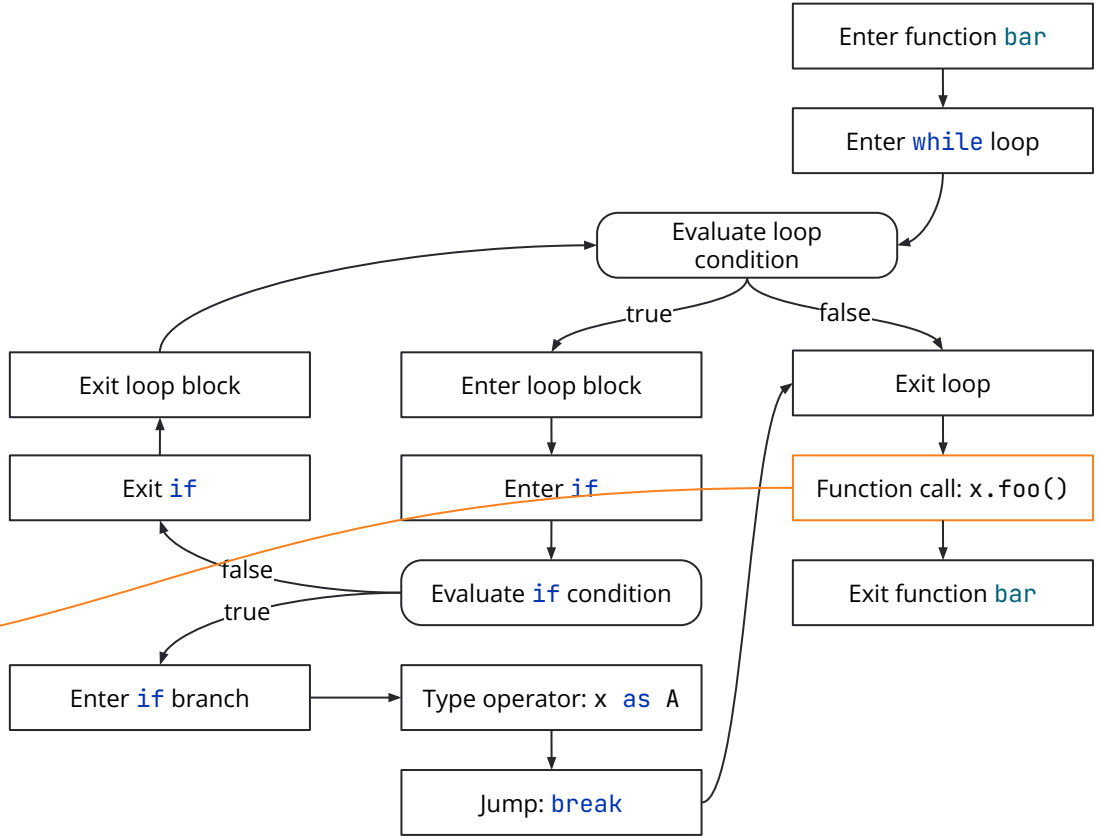
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



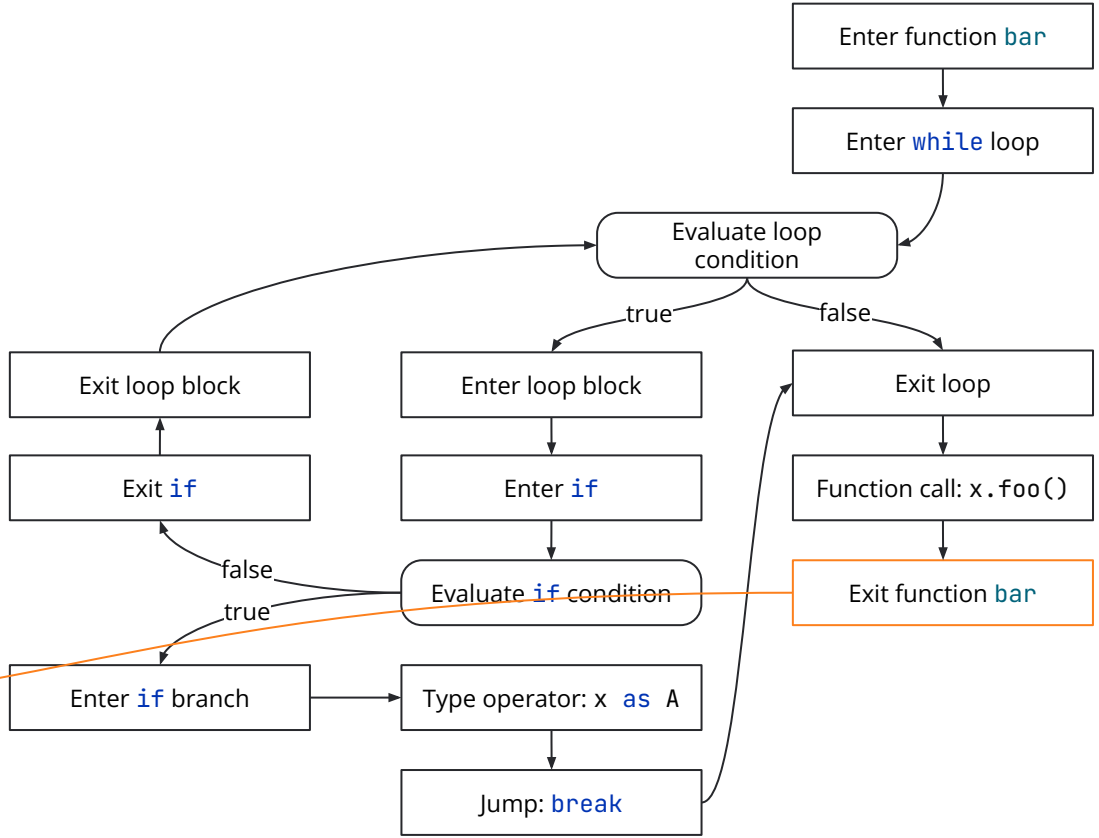
Kotlin compiler: control- and data-flow analysis

```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



Kotlin compiler: control- and data-flow analysis

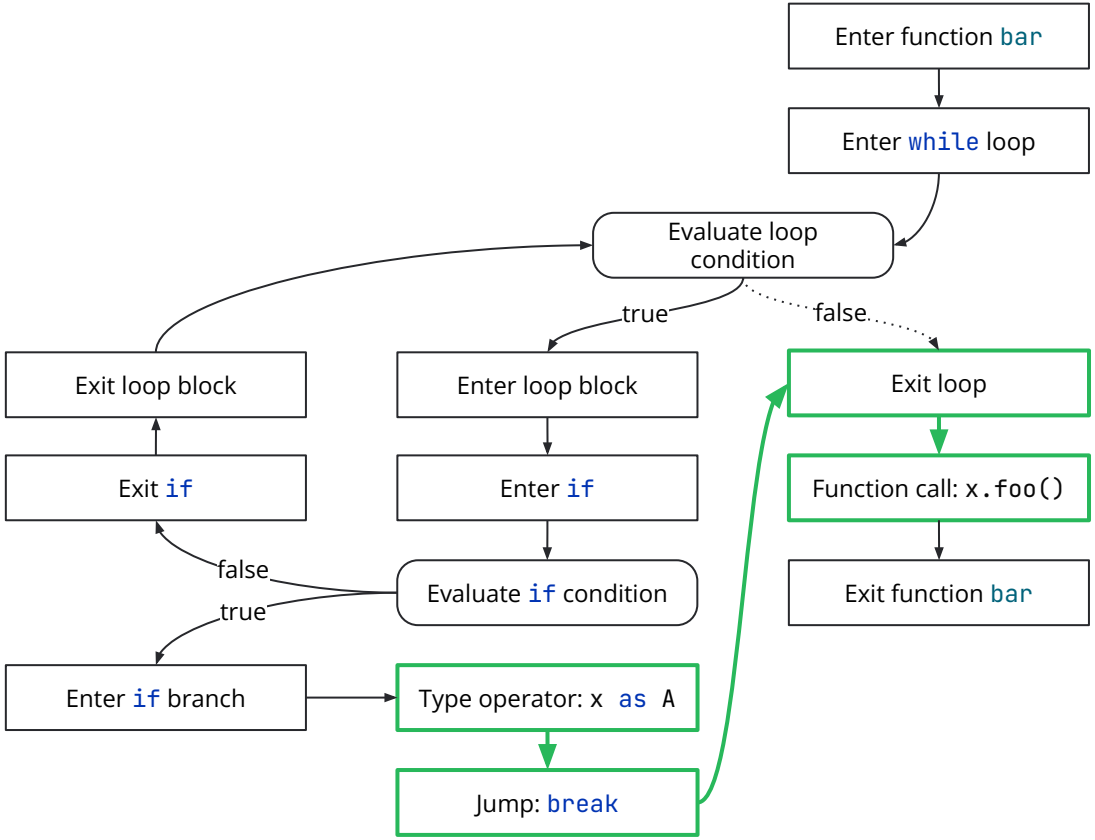
```kotlin
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```



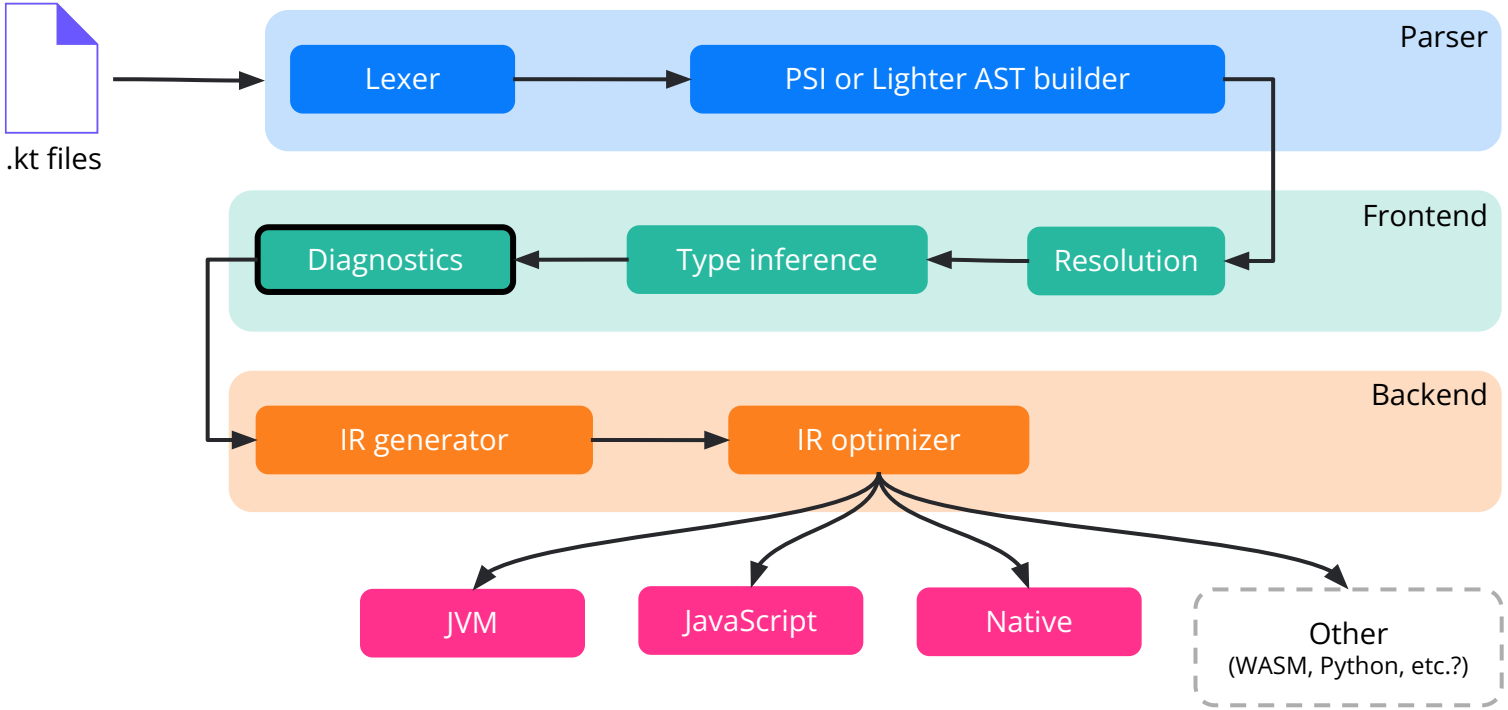
Kotlin compiler: control- and data-flow analysis

```
interface A {
    fun foo()
}

fun bar(x: Any, b: Boolean) {
    while (true) {
        if (b) {
            x as A
            break
        }
    }
    x.foo()
}
```

x is A



Kotlin compiler



Almost all checks you've ever seen in IntelliJ IDEA.

Kotlin compiler: diagnostics

```
open class A
open class B
open class C: A(), B()
```

Only one class may appear in a supertype list

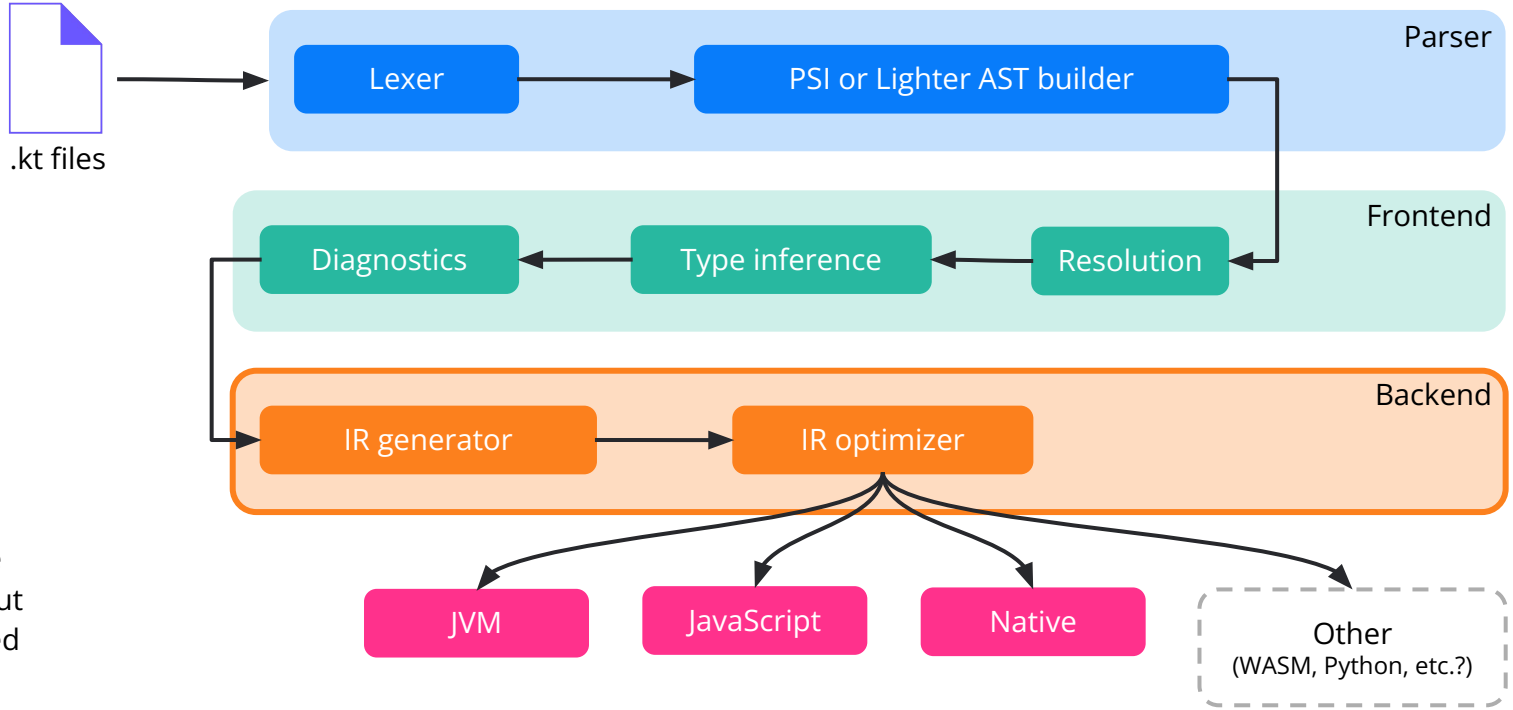
Remove supertype Alt+Shift+Enter More actions... Alt+Enter

```
public open class B
```

main.kt

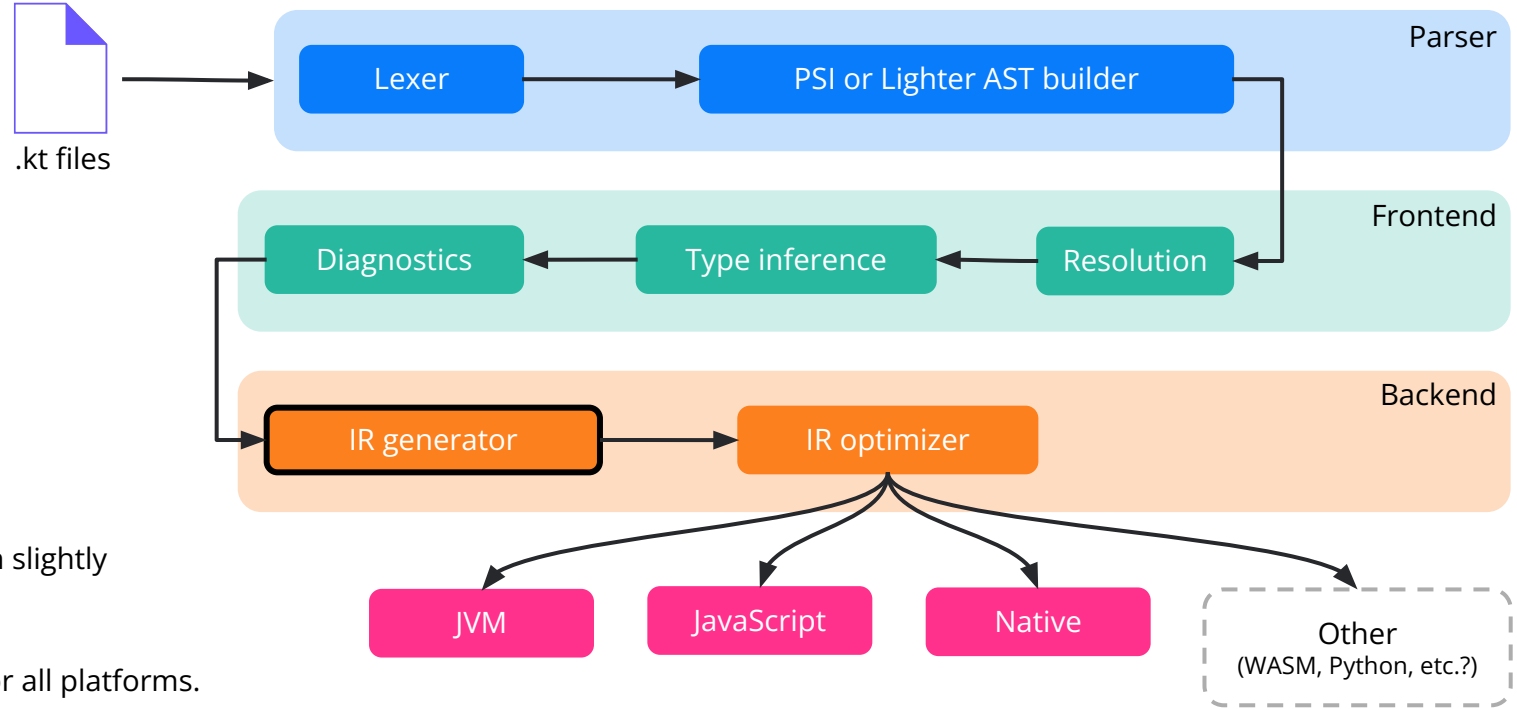
kotlinTest.wasmMain

Kotlin compiler



On the backend, we **DO NOT resolve**, but only use the received information

Kotlin compiler



IR, a representation slightly different from FIR.

It is still common for all platforms.

Kotlin compiler: resolved tree

```kotlin
fun hello(user: String) = println("Hello, $user")
```

Kotlin compiler: resolved tree

```
public fun hello(user: kotlin.String): kotlin.Unit
      defined in example in file Example.kt
```

```
fun hello(user: String) = println("Hello, $user")
```

Kotlin compiler: resolved tree

Reference to *value-parameter* *user*: kotlin.String
defined in example.hello

```
fun hello(user: String) = println("Hello, $user")
```

Kotlin compiler: resolved tree

Type: kotlin.String

```
fun hello(user: String) = println("Hello, $user")
```

Kotlin compiler: resolved tree

Type: kotlin.String

```
fun hello(user: String) = println("Hello, $user")
```

Kotlin compiler: resolved tree

Type: kotlin.Unit

```
fun hello(user: String) = println("Hello, $user")
```

Kotlin compiler: resolved tree

Call: kotlin.io.println(kotlin.String)

```
fun hello(user: String) = println("Hello, $user")
```

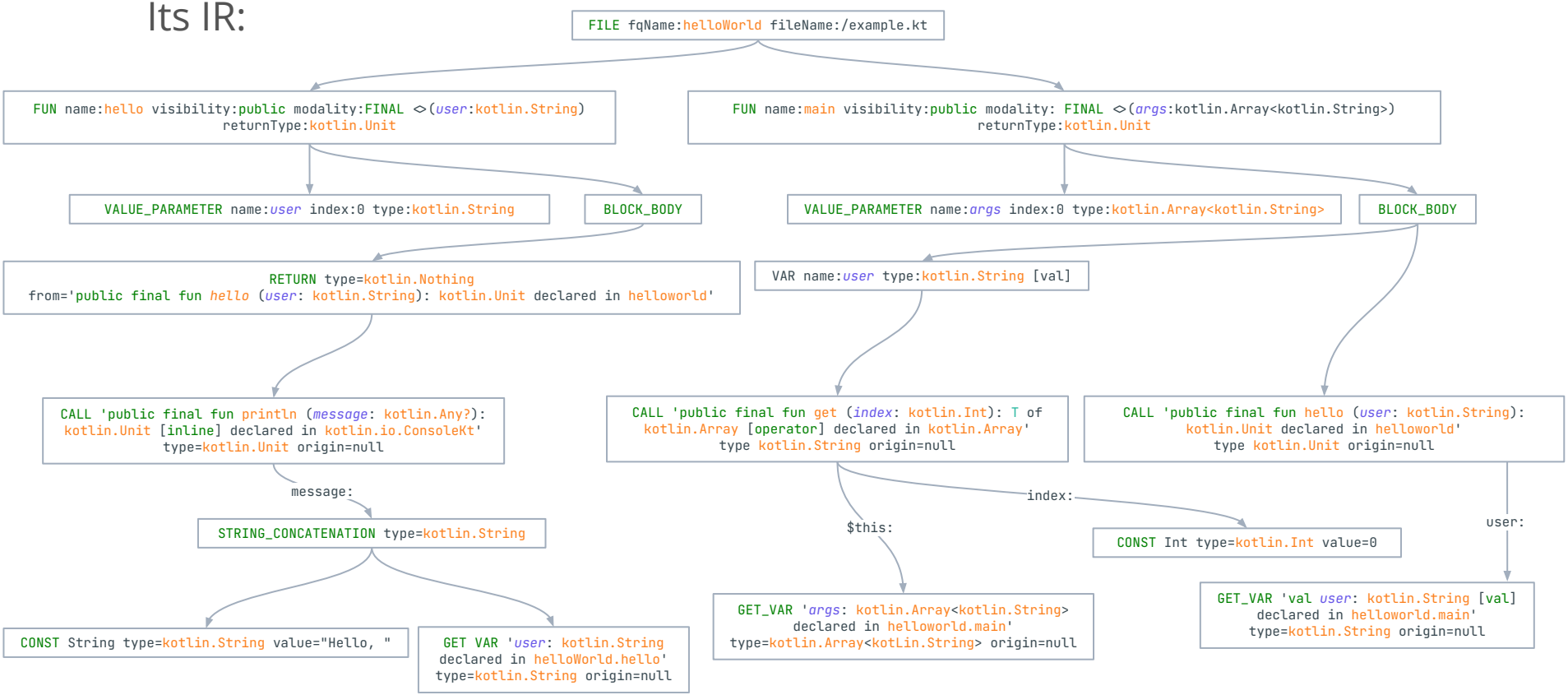
IR? Yet another tree!

The code:

```kotlin
// file 'src/kotlin/example.kt'

package helloWorld

fun hello(user: String) = println("Hello, $user")

fun main(args: Array<String>) {
   val user = args[0]
   hello(user)
}
```

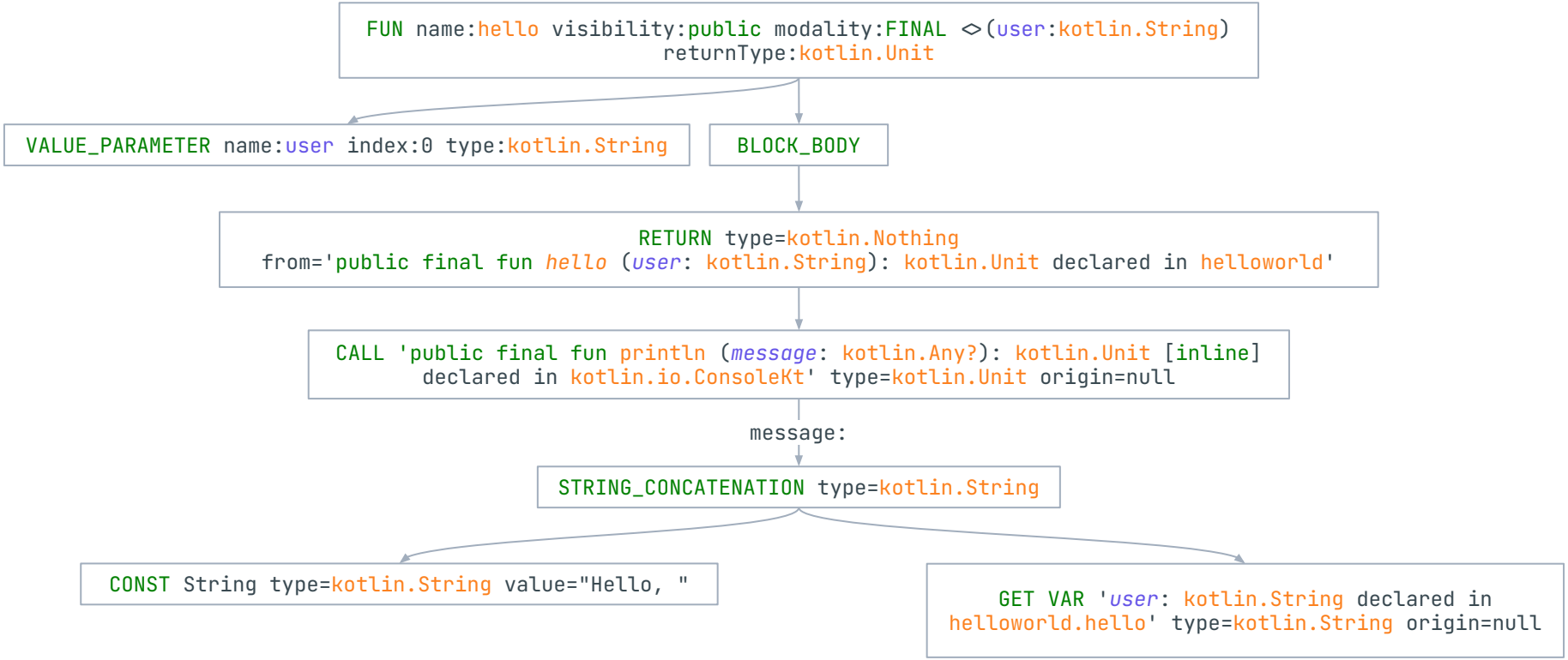
IR? Yet another tree!

Its IR:

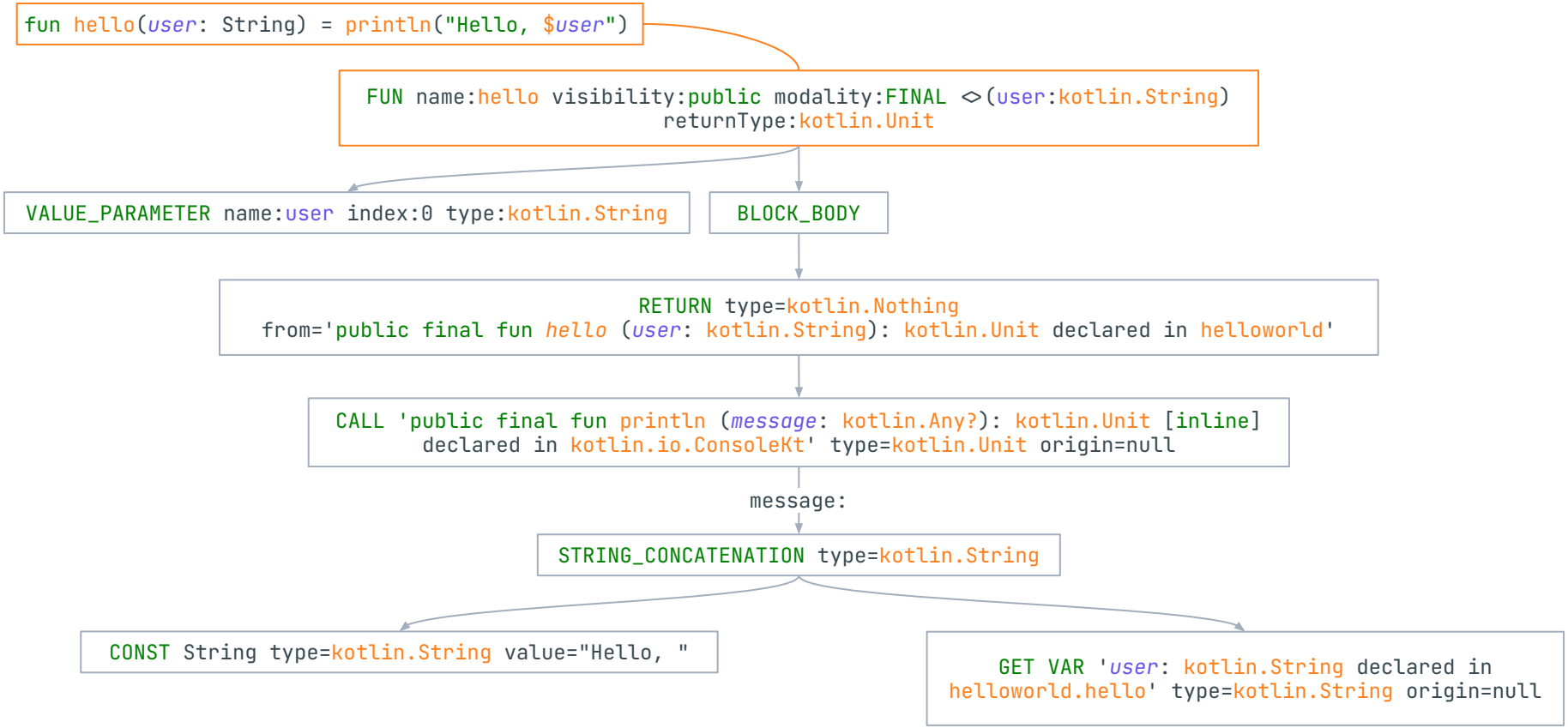


Back-end intermediate representation: a closer look

```
fun hello(user: String) = println("Hello, $user")
```



Back-end intermediate representation: a closer look



Back-end intermediate representation: a closer look

```
fun hello(user: String) = println("Hello, $user")
```

```
FUN name:hello visibility:public modality:FINAL <>(user:kotlin.String)
                                returnType:kotlin.Unit
```

```
VALUE_PARAMETER name:user index:0 type:kotlin.String
```

```
BLOCK_BODY
```

```
RETURN type=kotlin.Nothing
from='public final fun hello (user: kotlin.String): kotlin.Unit declared in helloworld'
```

```
CALL 'public final fun println (message: kotlin.Any?): kotlin.Unit [inline]
       declared in kotlin.io.ConsoleKt' type=kotlin.Unit origin=null
```

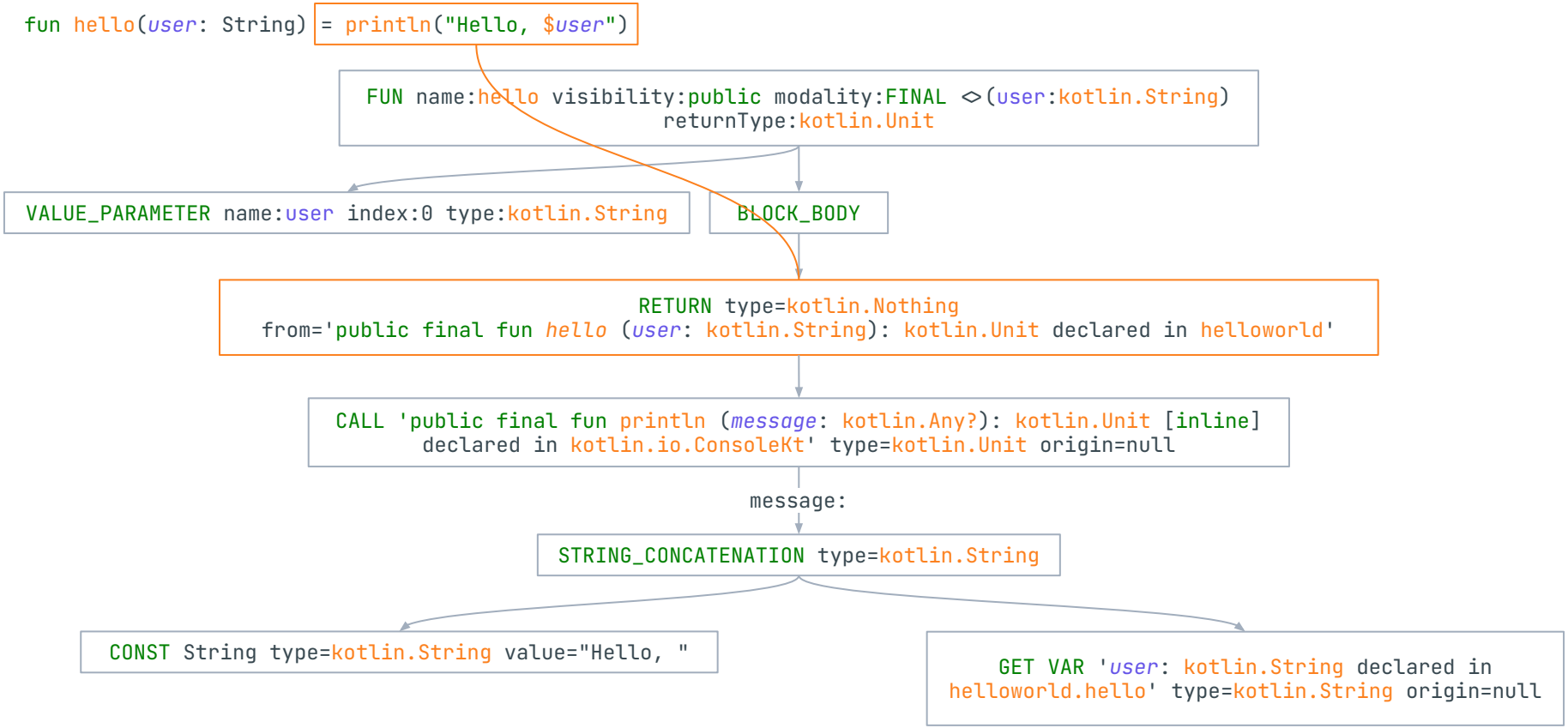
```
message:
```

```
STRING_CONCATENATION type=kotlin.String
```

```
CONST String type=kotlin.String value="Hello, "
```

```
GET VAR 'user: kotlin.String declared in
helloworld.hello' type=kotlin.String origin=null
```

Back-end intermediate representation: a closer look



Back-end intermediate representation: a closer look

```
fun hello(user: String) = println("Hello, $user")
```

```
FUN name:hello visibility:public modality:FINAL <>(user:kotlin.String)
                                returnType:kotlin.Unit
```

```
VALUE_PARAMETER name:user index:0 type:kotlin.String
```

```
BLOCK_BODY
```

```
RETURN type=kotlin.Nothing
from='public final fun hello (user: kotlin.String): kotlin.Unit declared in helloworld'
```

```
CALL 'public final fun println (message: kotlin.Any?): kotlin.Unit [inline]
      declared in kotlin.io.ConsoleKt' type=kotlin.Unit origin=null
```

```
message:
```

```
STRING_CONCATENATION type=kotlin.String
```

```
CONST String type=kotlin.String value="Hello, "
```

```
GET VAR 'user: kotlin.String declared in
helloworld.hello' type=kotlin.String origin=null
```

Back-end intermediate representation: a closer look

```
fun hello(user: String) = println("Hello, $user")
```

```
FUN name:hello visibility:public modality:FINAL <>(user:kotlin.String)
                                    returnType:kotlin.Unit
```

```
VALUE_PARAMETER name:user index:0 type:kotlin.String
```

```
BLOCK_BODY
```

```
RETURN type=kotlin.Nothing
from='public final fun hello (user: kotlin.String): kotlin.Unit declared in helloworld'
```

```
CALL 'public final fun println (message: kotlin.Any?): kotlin.Unit [inline]
        declared in kotlin.io.ConsoleKt' type=kotlin.Unit origin=null
```

```
message:
```

```
STRING_CONCATENATION type=kotlin.String
```

```
CONST String type=kotlin.String value="Hello, "
```

```
GET VAR 'user: kotlin.String declared in
helloworld.hello' type=kotlin.String origin=null
```

Back-end intermediate representation: a closer look

```
fun hello(user: String) = println("Hello, $user")
```

```
FUN name:hello visibility:public modality:FINAL <>(user:kotlin.String)
                                returnType:kotlin.Unit
```

```
VALUE_PARAMETER name:user index:0 type:kotlin.String
```

```
BLOCK_BODY
```

```
RETURN type=kotlin.Nothing
from='public final fun hello (user: kotlin.String): kotlin.Unit declared in helloworld'
```

```
CALL 'public final fun println (message: kotlin.Any?): kotlin.Unit [inline]
declared in kotlin.io.ConsoleKt' type=kotlin.Unit origin=null
```

```
message:
```

```
STRING_CONCATENATION type=kotlin.String
```

```
CONST String type=kotlin.String value="Hello, "
```

```
GET VAR 'user: kotlin.String declared in
helloworld.hello' type=kotlin.String origin=null
```

Back-end intermediate representation: a closer look

```
fun hello(user: String) = println("Hello, $user")
```

```
FUN name:hello visibility:public modality:FINAL <>(user:kotlin.String)
                                    returnType:kotlin.Unit
```

```
VALUE_PARAMETER name:user index:0 type:kotlin.String
```

```
BLOCK_BODY
```

```
RETURN type=kotlin.Nothing
from='public final fun hello (user: kotlin.String): kotlin.Unit declared in helloworld'
```

```
CALL 'public final fun println (message: kotlin.Any?): kotlin.Unit [inline]
        declared in kotlin.io.ConsoleKt' type=kotlin.Unit origin=null
```

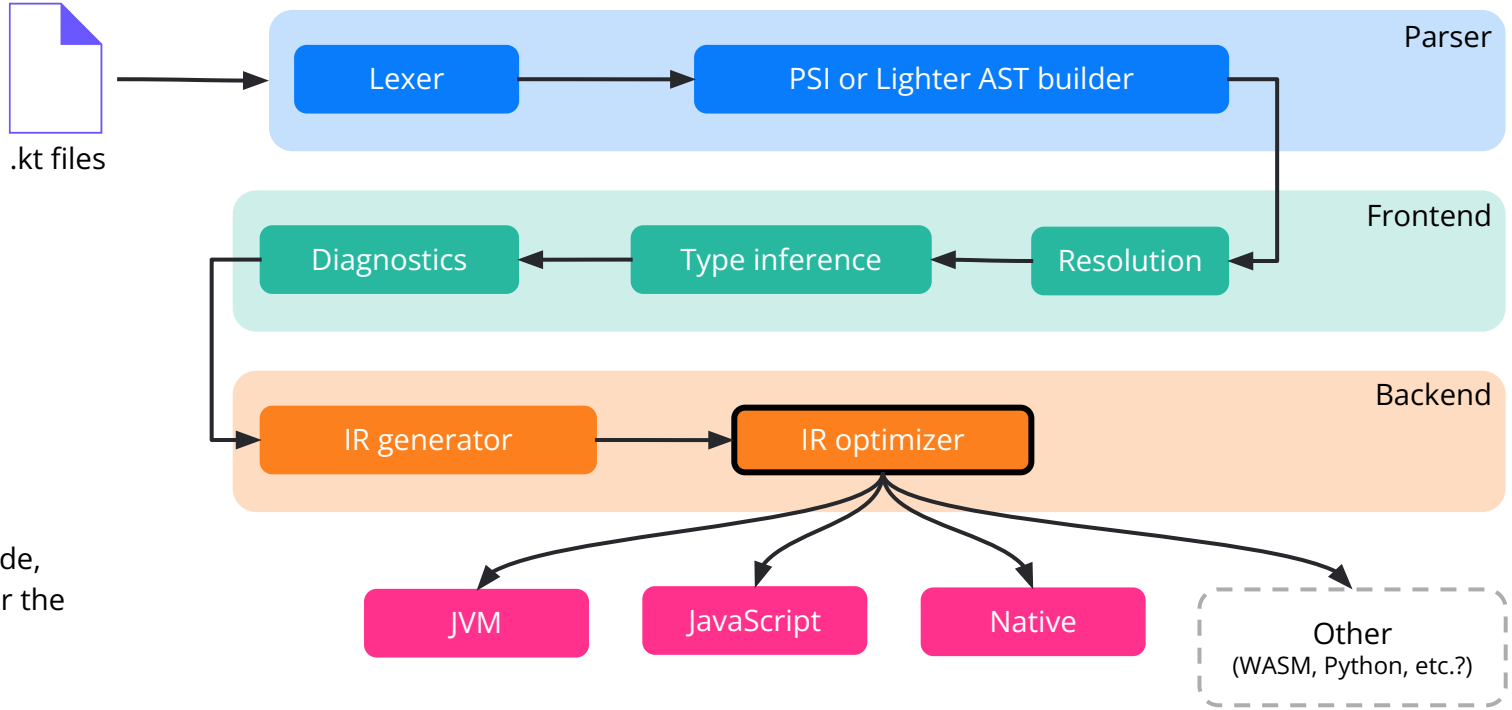
```
message:
```

```
STRING_CONCATENATION type=kotlin.String
```

```
CONST String type=kotlin.String value="Hello, "
```

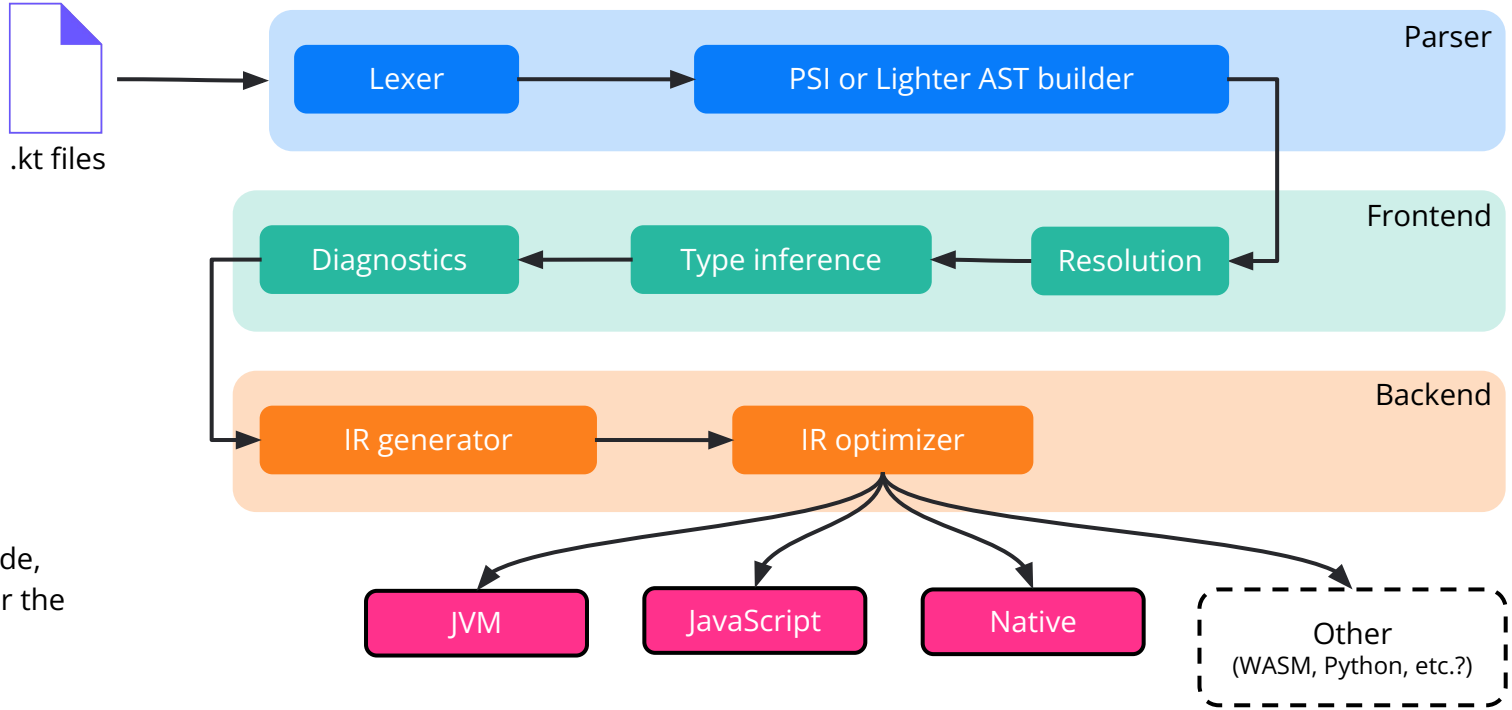
```
GET VAR 'user: kotlin.String declared in
helloworld.hello' type=kotlin.String origin=null
```

Kotlin compiler



Platform-specific code, such as bytecode for the JVM

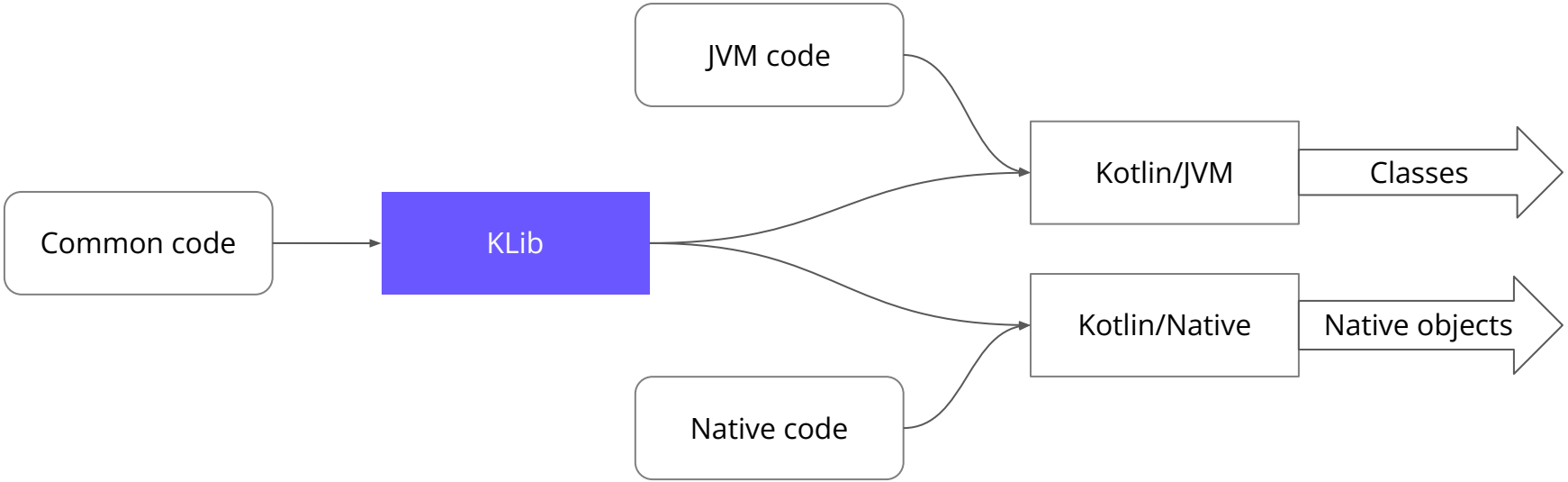
Kotlin compiler



Platform-specific code, such as bytecode for the JVM

KLibs

JAR analogues – store a serialized IR for the subsequent use of cross-platform libraries.



Compiler plugins

You can extend **any** compile phase via compiler plugins. To do so, you need to implement compiler extensions and register them.

To register extensions, you need to inherit from `CompilerPluginRegistrar` (`ComponentRegistrar` for previous versions).

Don’t forget to use `supportsK2 = true` if you need to support the FIR frontend

Compiler plugins: FIR extensions

To get the full list of the extensions, please see: `package org.jetbrains.kotlin.fir.extensions` (link).

Consider an example:

```
/*
 * Generates top level class
 *
 * package foo.bar
 *
 * public final class MyClass {
 *     fun foo(): String = "Hello world"
 * }
 */
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```kotlin
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```kotlin
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    companion object {
          // foo.bar.MyClass
        val MY_CLASS_ID = ClassId(
            FqName.fromSegments(listOf("foo", "bar")),
            Name.identifier("MyClass")
        )
          // foo.bar.MyClass.foo
        val FOO_ID = CallableId(MY_CLASS_ID, Name.identifier("foo"))
    }
    override fun generateClassLikeDeclaration(classId: ClassId): FirClassLikeSymbol<*>? {
        ...
    }
    ...
}
```

Compiler plugins: FIR extensions

You use a special key to mark **everything** generated by the compiler and can transfer **any** information between frontend and backend. So it also helps to find the new declaration to generate it's IR.

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    object Key : GeneratedDeclarationKey()
...
}
```

Compiler plugins: FIR extensions

Don’t forget to register all new declarations.

```
class SimpleClassGenerator(session: FirSession) : FirDeclarationGenerationExtension(session) {
    ...
    override fun getTopLevelClassIds(): Set<ClassId> {
        return setOf(MY_CLASS_ID)
    }

    override fun hasPackage(packageFqName: FqName): Boolean {
        return packageFqName == MY_CLASS_ID.packageFqName
    }

}
```

Compiler plugins: IR extensions

Actually, the compiler has only one extension for IRs: `IrGenerationExtension`.

You just need to implement some *transformers* and accept them:

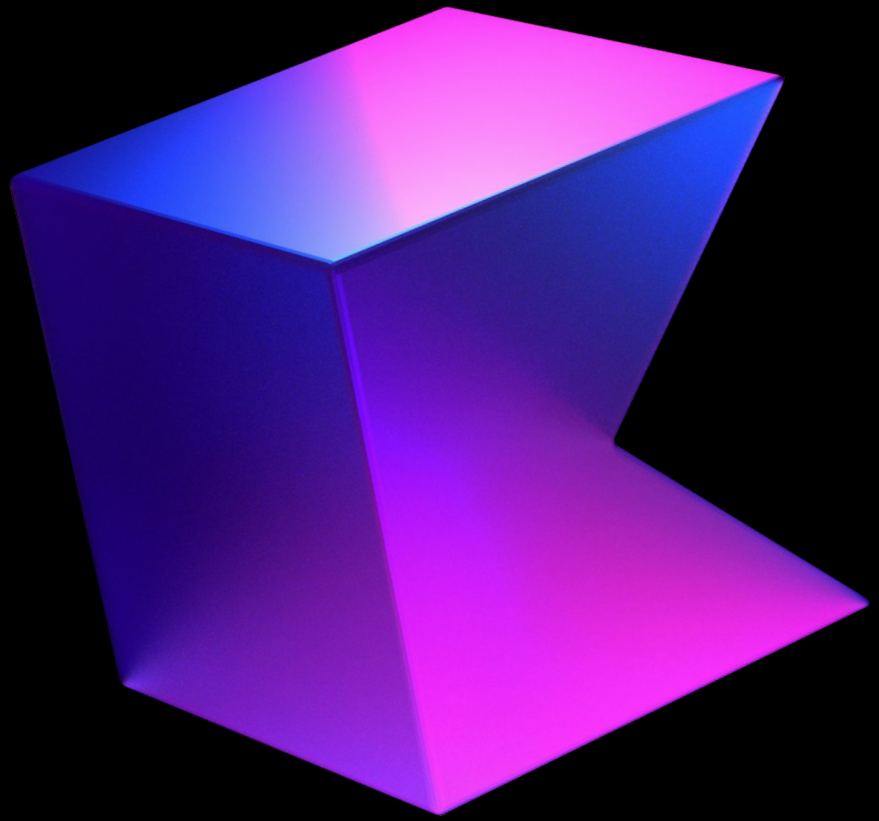
```
class SimpleIrGenerationExtension: IrGenerationExtension {
    override fun generate(moduleFragment: IrModuleFragment, pluginContext: IrPluginContext) {
        val transformers = listOf(SimpleIrBodyGenerator(pluginContext))
        for (transformer in transformers) {
            moduleFragment.acceptChildrenVoid(transformer)
        }
    }
}
```

Don’t forget to check the `key` (use the `interestedIn` function)!

Compiler plugins: popular plugins

- kotlinx.serialization — Generates visitor code for serializable classes.
- all-open — Marks all classes as open classes.
- kapt — An annotation processor.
- ksp — An API for developing lightweight compiler plugins.
- Jetpack Compose — Generates efficient UI from its declarative description.
- Arrow Meta — Compiler plugin API that empowers all Arrow libraries.
- ...

Thanks!



@kotlin | Developed by JetBrains